Academic submission

MedChain

Blockchain-Based Patient Consent Management and Secure Medical History Sharing

Algorithm 08 — Patient Consent Management Contract

Implementation: Next.js 16 (App Router), React 19, TypeScript, Tailwind CSS, Web Crypto API (SHA-256), localStorage persistence.
Evidence in this document was captured from the running application on 06 Nov 2026, 05:49:42. Every event ID, transaction ID and hash shown is taken from that run; nothing is typed in by hand.

The blockchain network, smart-contract deployment and credential signatures are simulated in the browser, as allowed for the initial demonstration. The access-decision logic and the SHA-256 hash chain are real and execute as described. All patient data is synthetic.

Contents

1. Objective
2. Running the application
3. Users, roles and sample data
4. Algorithm 08 — logic and reasoning
5. Hash-chained ledger — design and reasoning
6. Case 1 — successful ECG request (approved), step by step with the live ledger
7. Case 2 — MRI request outside consent (denied by Rule 2)
8. Further denial cases — unauthorized hospital (Rule 1), revoked consent and expired consent (Rule 3)
9. Patient Portal — consent and access history
10. Ledger integrity — verification, tampering and hash inspection
11. Smart Contract Studio
12. Requirement compliance matrix
13. Test results summary
14. Appendix A — full hash chain of the approved transaction
15. Appendix B — contract code as deployed

1. Objective

MedChain demonstrates how a smart contract can control access to a patient's medical history when it is shared between hospitals. A record is released only when **all** of these hold:

- the requesting healthcare professional is authenticated, verified and authorized (Rule 1);
- the patient has granted consent to that hospital or professional, and the requested records are within the consented scope (Rule 2);
- the request's purpose matches the consent, and the consent is active, not expired and not revoked (Rule 3);
- and the source hospital approves the request.

Every action in the process is written to a SHA-256 hash-chained ledger that is visible live on the right side of the screen.

Patient grants consent → Physician authenticates → History requested → Source hospital receives and verifies → Algorithm 08: R1, R2, R3 → Hospital responds → Access granted / denied → Records released → Ledger

2. Running the application

```
npm install
npm run dev
# open http://localhost:3000 (window at least 1200 px wide so the live ledger stays on the right)
```

The application has four screens, matching the assignment:

Screen	Purpose
Request Workflow	The complete process on one page in six numbered steps. Each step states who acts; a "Sign in as …" button appears when the next step needs a different person.
Patient Portal	Profile, record categories, grant() and revoke() of consent, active permissions, requests for the patient's records with decisions, consent and access history, demo clock (for the expiry test).
Smart Contract Studio	Contract ID, version and SHA-256 hash; the three fixed rules; the pseudocode and the actual TypeScript executed; execution history; simulated deployment receipt.
Transaction Ledger	All events with search and filters, transaction history, expandable details with hash inspection, Verify Ledger, tampering demonstration, CSV export, Reset Demo Data.

Layout. Left: navigation. Centre: the workspace. Right: the **live transaction ledger**, which shows each event as soon as it has been hashed and saved: event ID, transaction ID, request ID, event type, actor and role, requesting and source hospitals, patient, record categories, purpose, timestamp, status, previous and current hash. The newest event is highlighted, and each event expands to its full details.

3. Users, roles and sample data

Demo user	Role	Hospital	Used for
Ayesha Khan	Patient	—	Grants and revokes consent; views requests and decisions
Dr. Ahmed Raza	Physician	H1 City Hospital	Authorized requester
Dr. Victor Hale	Physician	H5 External Hospital (not registered)	Unauthorized requester for the Rule 1 test
Sara Malik	Hospital Records Officer	H2 General Hospital	Receives, verifies, runs the contract, approves or rejects, releases
Imran Qureshi	Hospital Administrator	H3 Medical Centre	Creates and deploys contract versions; cannot override rules
Grace Mensah	Auditor	—	Verifies the ledger

Patient: Ayesha Khan (PAT-001) with synthetic LAB, ECG and MRI records held at H2 General Hospital.

Hospitals: H1 City Hospital, H2 General Hospital, H3 Medical Centre, H4 Central Hospital (registered and active); H5 External Hospital (not registered, unauthorized).

Initial consent (CON-0001): authorized hospital City Hospital (H1); permitted records LAB and ECG; MRI not permitted; purpose TREATMENT; expiry 31 December 2026; status ACTIVE.

Why these users: one user per role named in the assignment, plus one unauthorized physician so that Rule 1 can be shown failing on real data rather than on a hard-coded flag.

4. Algorithm 08 — Patient Consent Management Contract

The algorithm is one TypeScript module, `src/lib/contract.ts`, with three public functions. Every screen calls these functions through the central store; there is no second copy of the decision logic. The pure functions take their inputs as arguments, so the same inputs always give the same decision.

4.1 grant(patient, grantee, scope, purpose, expiry)

```
require patient has a registered DID
require scope is not empty
require now < expiry <= now + maxConsentDays          (policy: 365 days)
mark any ACTIVE consent of this patient for this grantee as SUPERSEDED
add Consent(patient, grantee, scope, purpose, expiry, ACTIVE)
record CONSENT_GRANTED on the ledger
```

Reasoning. The grantee is either a hospital or one designated professional. Superseding the old consent keeps exactly one active consent per grantee, so check() never has to choose between conflicting consents. The expiry limit stops open-ended sharing.

4.2 revoke(patient, grantee)

```
require the caller is the patient
require at least one ACTIVE consent exists for (patient, grantee)
set those consents to REVOKED, store revokedAt = now
record CONSENT_REVOKED on the ledger
```

Reasoning. The consent is kept (with status REVOKED) instead of being deleted, so the audit trail stays complete and Rule 3 can explain *why* a later request fails.

4.3 check(patient, grantee, record_type)

```
consent = newest ACTIVE consent for (patient, grantee.hospital or grantee.professional),
          otherwise the newest consent of any status
if not R1(grantee, processor)                    → DENY (Rule 1)
if not R2(consent, grantee, record_type)         → DENY (Rule 2)
if not R3(consent, request.purpose, now)         → DENY (Rule 3)
→ APPROVE   (records are still only released after the source hospital approves)
```

Why this order. Identity is checked first so that an unauthorized party is stopped before the contract looks at the patient's consent at all. Scope (R2) is checked before validity (R3) because a request for records the patient never consented to should be reported as out of scope, which is the more useful explanation. Evaluation stops at the first failure, like `require()` in a smart contract.

Why the newest consent of any status is used when none is active. If a consent was revoked or expired, Rule 3 can then say exactly that ("consent CON-0001 is REVOKED") instead of the less precise "no consent found".

4.4 The three fixed rules

Rule	Condition	Checks (actual inputs shown on screen)	Why it exists
R1 Identity and hospital authorization	`user.authenticated && user.credentialVerified && hospital.status == ACTIVE && role.hasPermission`	Requester signed in; requester credential verified (trusted issuer, unaltered, in date, not revoked); requesting hospital ACTIVE; role Physician has request:create; the processing records officer is signed in, verified, has request:process and works at the source hospital; source hospital ACTIVE.	Only known, verified professionals at registered hospitals may request or process patient data.

Rule	Condition	Checks (actual inputs shown on screen)	Why it exists
R2 Patient consent and record scope	`consent != null && grantee IN consent.grantees && requestedRecordType IN consent.allowedScope`	A consent exists for the requesting hospital or the designated professional; every requested category is in the consented scope.	The patient decides who sees which records (data minimisation).
R3 Purpose, expiry and revocation	`request.purpose == consent.purpose && consent.status == ACTIVE && currentTime < consent.expiry`	Purpose matches; consent ACTIVE (not revoked or superseded); current time before expiry.	Consent is purpose-bound and time-limited, and the patient can withdraw it at any time.

4.5 No override, and release only after hospital approval

- A failed rule records CONTRACT_DENIED and ACCESS_DENIED and ends the request. The screens then show no Approve or Release action, and the store itself refuses approval unless the request's status is CONTRACT_APPROVED. No role, including the Hospital Administrator, has an override function.
- The baseline rules are fixed. Contract Studio validation rejects any new version whose rule text differs from the fixed rules.
- Release requires both: check() returned APPROVE **and** the source hospital approved. Only records that are both requested and inside the consent are released.

5. Hash-chained ledger — design and reasoning

Every meaningful action is written through one function, `recordLedgerEvent()`, inside a single queued commit (`src/lib/store.tsx`):

1. The action changes the application state and records its events with the actual actor, role, hospital, patient and request.
2. Each new event gets an event ID; events of the same request reuse the request's transaction ID.
3. Each event is hashed in order: **`currentHash = SHA256(previousHash + canonicalEventPayload)`**, using the browser's Web Crypto API.
4. The new state is saved to localStorage. Only then does any screen, including the live ledger, update. If saving fails, the action is rejected and reported.

Design choice	Reason
Genesis previousHash = 64 zeros	The first event has nothing before it; a fixed starting value makes the chain verifiable from the start.
Canonical payload (JSON with sorted keys, hash fields excluded)	The same event always produces the same text, and so the same hash, on any machine.
Sequential processing queue	Two quick actions can never compute their hashes from the same previous hash (no race in the chain).
Hash before display, persist before display	No event is ever shown as recorded unless it has been hashed and saved.
Shared state across browser tabs	A write always builds on the newest saved ledger, so one tab cannot overwrite events written by another.
Verification recomputes every hash	Changing any stored field changes that event's recomputed hash, so verification reports the exact event that was altered.

Fields of every event: Event ID, Transaction ID, Request ID, timestamp, actor, actor role, hospital, event type, execution result, explanation, contract version, patient and request context (patient, requesting hospital, source hospital, categories, purpose, reason), previousHash, currentHash.

Event types used: AUTHENTICATION_VERIFIED, CREDENTIAL_VERIFIED, HOSPITAL_VERIFIED, CONSENT_GRANTED, CONSENT_REVOKED, REQUEST_CREATED, REQUEST_RECEIVED, REQUEST_REVIEW_STARTED, HOSPITAL_VERIFICATION_COMPLETED, CONTRACT_EXECUTION_STARTED, RULE_1_EVALUATED, RULE_2_EVALUATED, RULE_3_EVALUATED, CONTRACT_APPROVED, CONTRACT_DENIED, HOSPITAL_APPROVED, HOSPITAL_REJECTED, ACCESS_GRANTED, ACCESS_DENIED, MEDICAL_RECORD_RELEASED, plus LEDGER_VERIFIED and the genesis events.

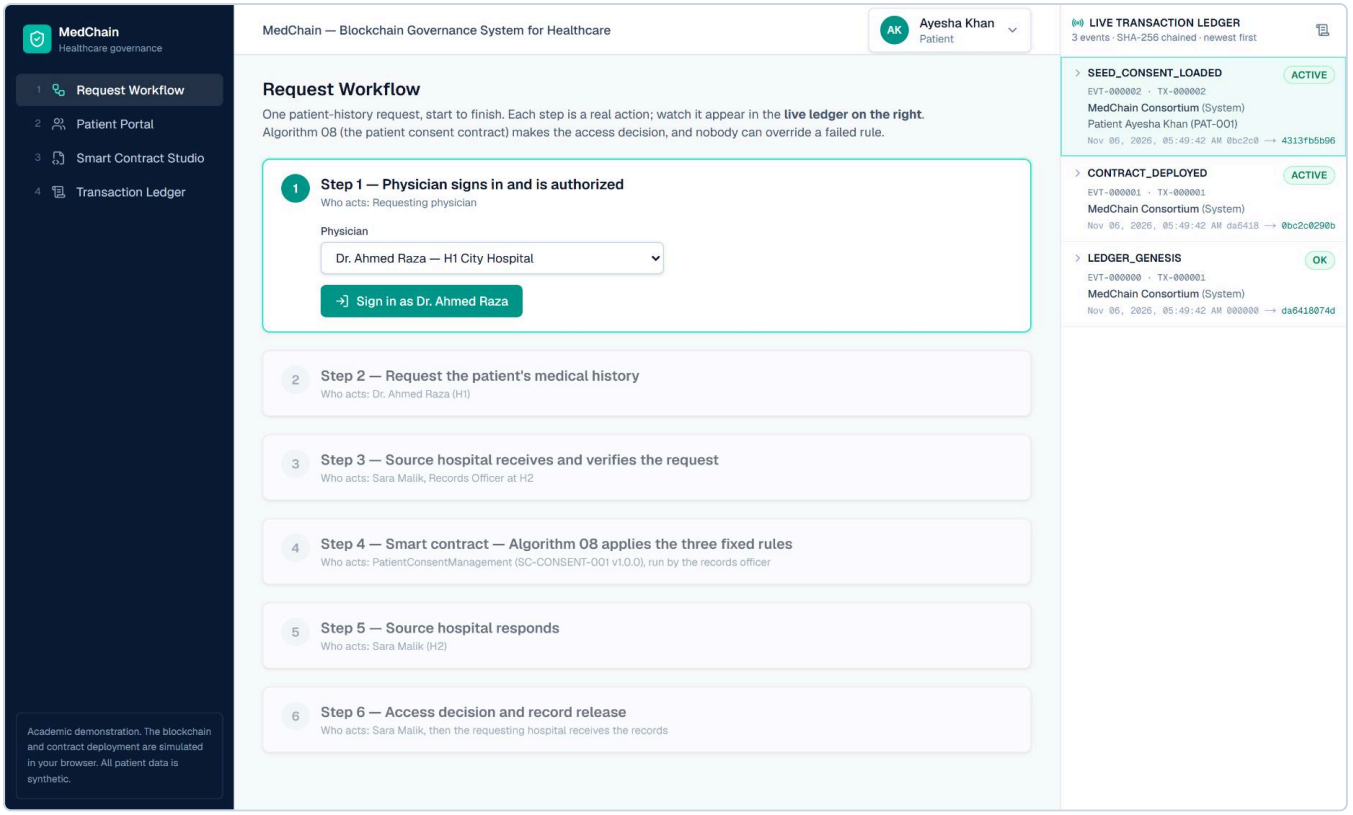


Figure 1. Starting point after Reset Demo Data: three genesis events in the live ledger (right). The first has previousHash 000000…

Genesis events

Event / Tx / Request	Event type and explanation (as recorded)	Actor (role)	Result	previousHash → currentHash
EVT-000000 TX-000001	**LEDGER_GENESIS** Genesis event. Its previousHash is 64 zeros; every later event links to the one before it.	MedChain Consortium System	OK	000000000000… ↓ **da6418074d14…**
EVT-000001 TX-000001	**CONTRACT_DEPLOYED** PatientConsentManagement (SC-CONSENT-001) v1.0.0 deployed with the three fixed rules (simulated deployment).	MedChain Consortium System	ACTIVE	da6418074d14… ↓ **0bc2c0290b7c…**
EVT-000002 TX-000002	**SEED_CONSENT_LOADED** Initial demonstration data (not a user action): Ayesha Khan (PAT-001) allows H1 City Hospital to access LAB and ECG for Treatment until 2026-12-31.	MedChain Consortium System	ACTIVE	0bc2c0290b7c… ↓ **4313fb5b966b…**

6. Case 1 — successful ECG request (approved)

Dr. Ahmed Raza (City Hospital, H1) requests Ayesha Khan's ECG from General Hospital (H2) for treatment. The consent allows LAB and ECG for H1, so all rules should pass. Request **REQ-2026-0001**, transaction **TX-000004**. Each step below shows the screen at that moment, with the live ledger on the right, and the events that step wrote.

Step 1 — Physician signs in and is authorized

> **Assignment requirement:** A healthcare professional logs in; the system verifies identity, role, credentials and hospital registration.

What the user does: Selects Dr. Ahmed Raza and clicks *Sign in*.

What the system checks (logic): The credential is verified (issuer is a trusted consortium hospital, proof hash matches the credential content, not expired, not revoked), the hospital's registry status is checked, and the role's permissions are read.

Why: These are the facts Rule 1 will rely on, so they are established and recorded when the user signs in.

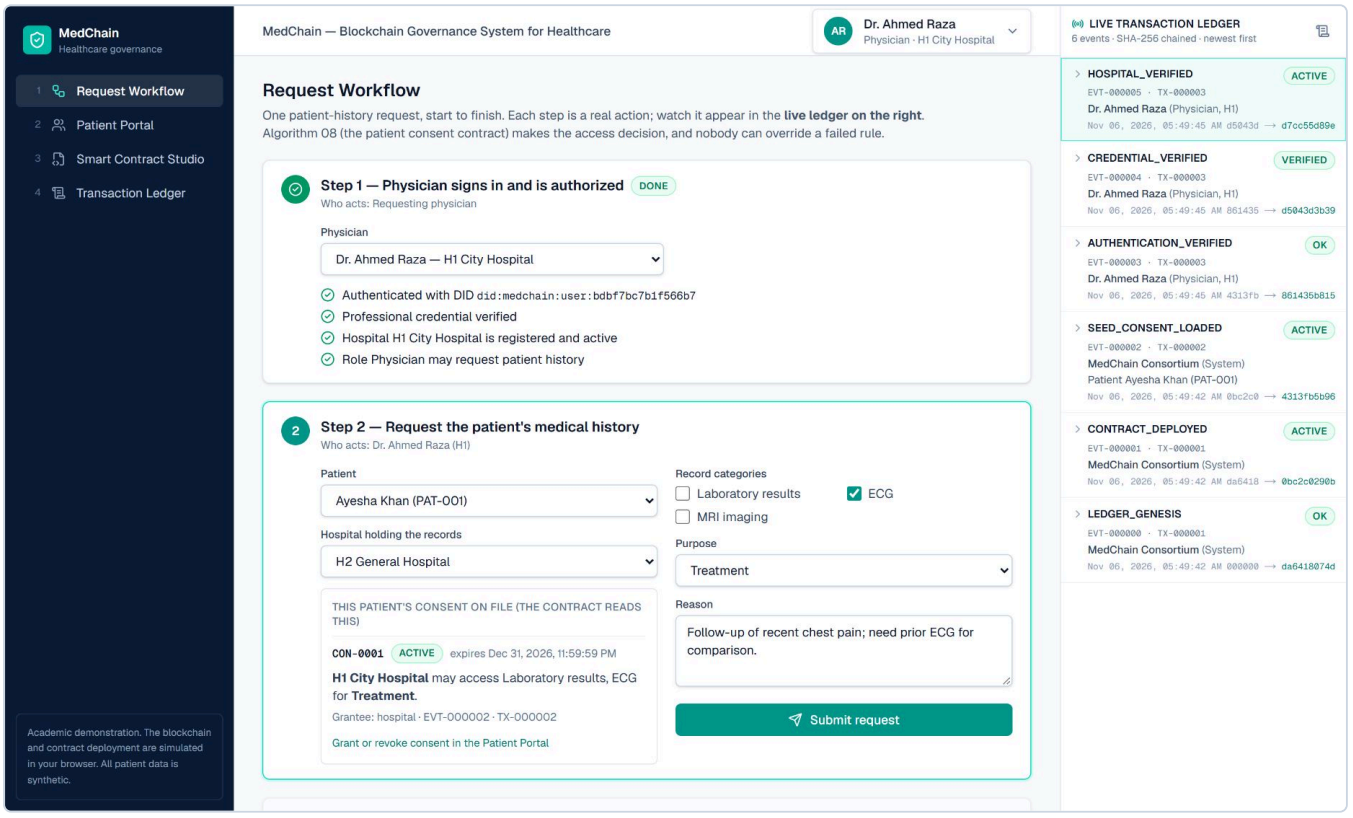


Figure 2. Step 1 complete: four green checks; three events appear at the top of the live ledger.

Live ledger: events written by this step (3)

Event / Tx / Request	Event type and explanation (as recorded)	Actor (role)	Result	previousHash → currentHash
EVT-000003 TX-000003	**AUTHENTICATION_VERIFIED** Dr. Ahmed Raza signed in as Physician using DID did:medchain:user:bdbf7bc7b1f566b7 (demo authentication).	Dr. Ahmed Raza Physician, H1	OK	4313fb5b966b… ↓ **861435b8151d…**
EVT-000004 TX-000003	**CREDENTIAL_VERIFIED** Credential urn:vc:medchain:u-phy-h1 verified: trusted issuer, intact, in date, not revoked.	Dr. Ahmed Raza Physician, H1	VERIFIED	861435b8151d… ↓ **d5043d3b3912…**

Event / Tx / Request	Event type and explanation (as recorded)	Actor (role)	Result	previousHash → currentHash
EVT-000005 TX-000003	**HOSPITAL_VERIFIED** H1 City Hospital is registered and active in the consortium registry.	Dr. Ahmed Raza Physician, H1	ACTIVE	d5043d3b3912… ↓ **d7cc55d89e89…**

Step 2 — Request the patient's medical history

> **Assignment requirement:** The physician selects a patient, the medical-history categories, the purpose and reason, and submits to the source hospital; a unique request ID and transaction reference are generated.

What the user does: Chooses Ayesha Khan, ECG, Treatment, a reason and source hospital H2, then clicks *Submit request*. The patient's consent on file is shown next to the form.

What the system checks (logic): Only a signed-in user whose role has request:create can submit. The request is placed in H2's inbox in the same saved change that records REQUEST_CREATED.

Why: The transaction ID ties every later event of this request together in the ledger.

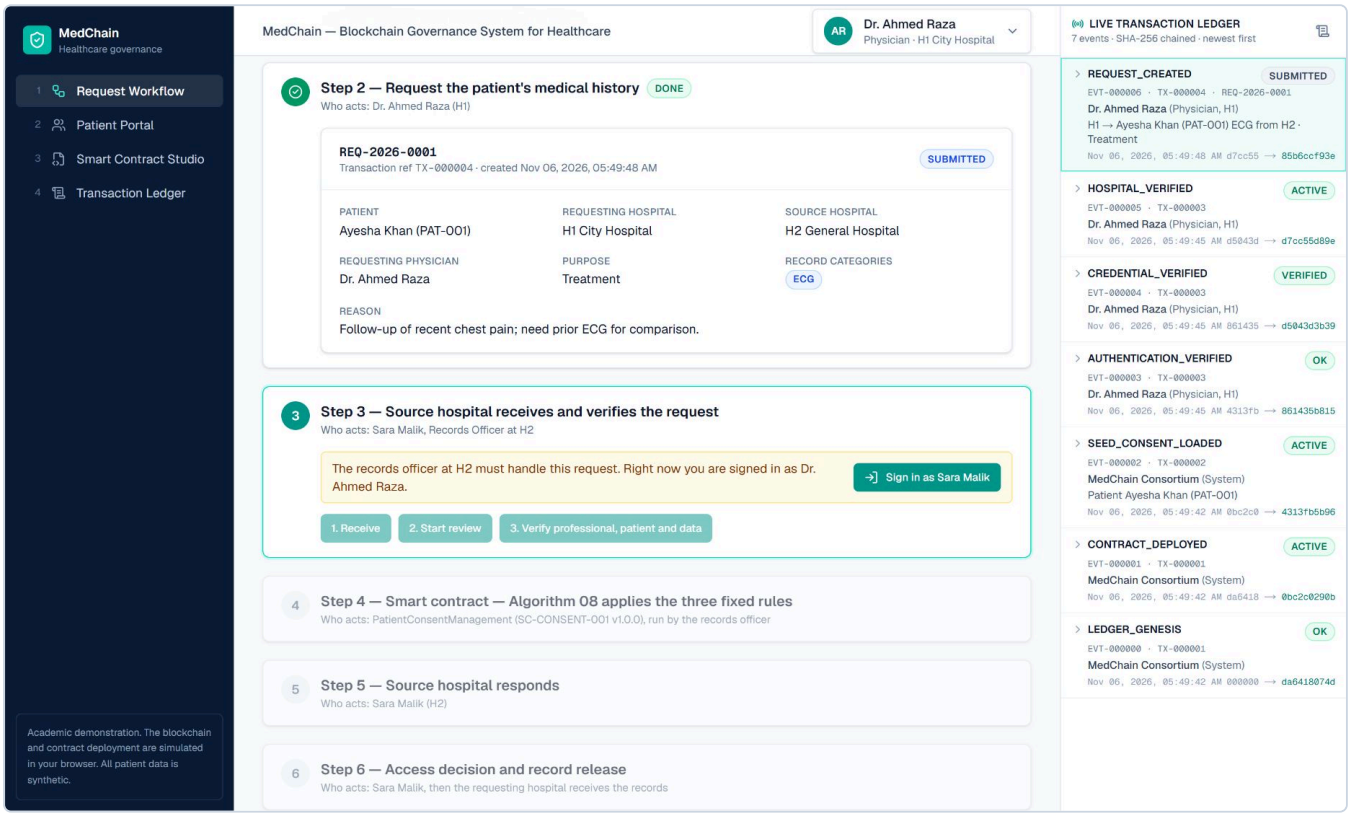


Figure 3. Request REQ-2026-0001 (transaction TX-000004) created. The live ledger shows who requested which patient's ECG, from which hospital and why.

Live ledger: events written by this step (1)

Event / Tx / Request	Event type and explanation (as recorded)	Actor (role)	Result	previousHash → currentHash
EVT-000006 TX-000004 REQ-2026-0001	**REQUEST_CREATED** Dr. Ahmed Raza (H1) requested Ayesha Khan's ECG from H2 for Treatment. Reason: Follow-up of recent chest pain; need prior ECG for comparison.	Dr. Ahmed Raza Physician, H1	SUBMITTED	d7cc55d89e89… ↓ **85b6ccf93e47…**

Step 3 — Source hospital receives and verifies the request

> **Assignment requirement:** The source hospital receives the request; an authorized Records Officer reviews it and verifies the patient details, requesting hospital, physician and requested records.

What the user does: Signs in as Sara Malik (Records Officer, H2), then clicks *Receive*, *Start review* and *Verify*.

What the system checks (logic): Verification checks: patient reference (DID registered), requesting hospital status, requesting physician's credential, whether H2 holds the requested records, and whether a usable consent is on file.

Why: This is the hospital's own check. It cannot approve anything by itself; the contract still decides.

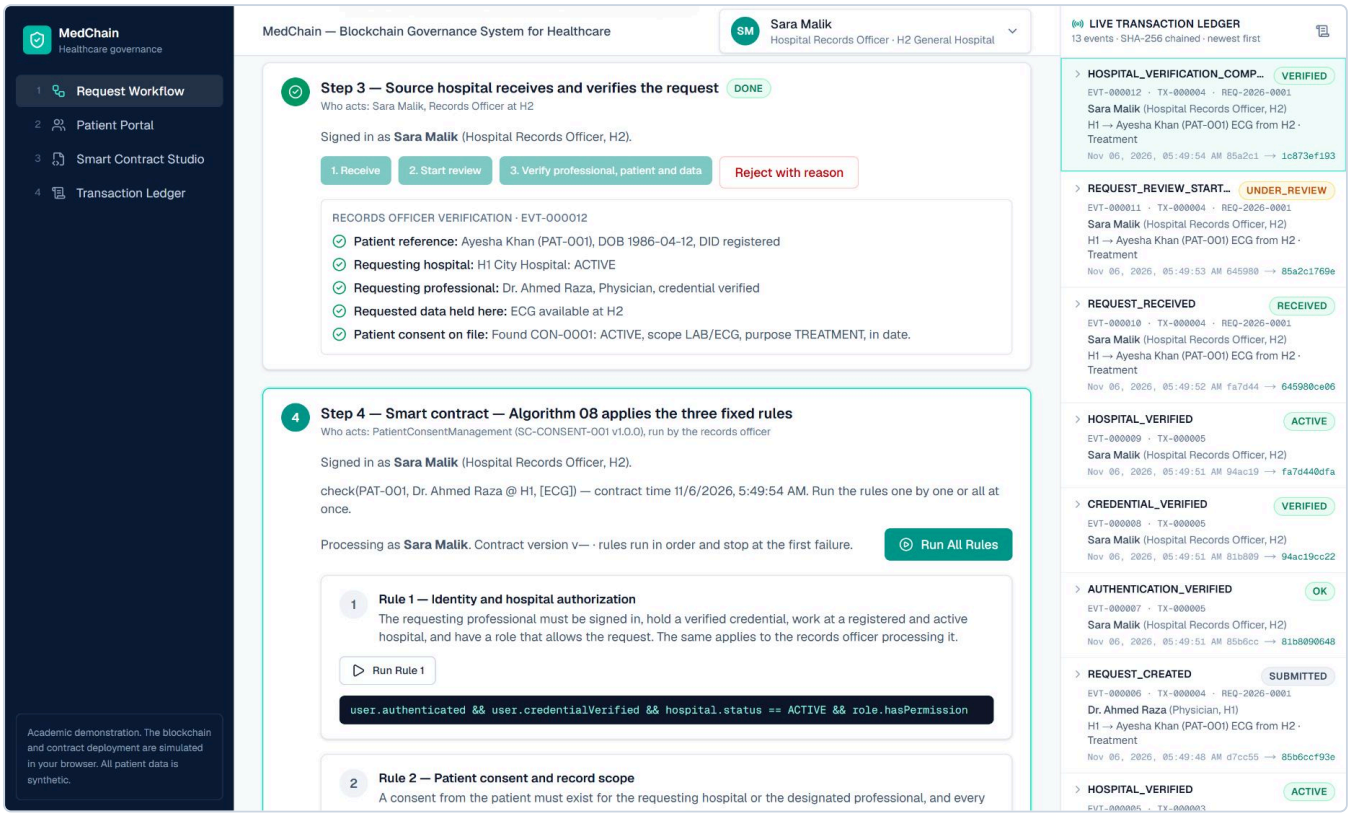


Figure 4. Step 3: receipt, review and verification recorded; verification checklist shown.

Live ledger: events written by this step (6)

Event / Tx / Request	Event type and explanation (as recorded)	Actor (role)	Result	previousHash → currentHash
EVT-000007 TX-000005	**AUTHENTICATION_VERIFIED** Sara Malik signed in as Hospital Records Officer using DID did:medchain:user:66ab9eea1309ceb0 (demo authentication).	Sara Malik Hospital Records Officer, H2	OK	85b6ccf93e47… ↓ **81b8090648cc…**
EVT-000008 TX-000005	**CREDENTIAL_VERIFIED** Credential urn:vc:medchain:u-rec-h2 verified: trusted issuer, intact, in date, not revoked.	Sara Malik Hospital Records Officer, H2	VERIFIED	81b8090648cc… ↓ **94ac19cc22fc…**
EVT-000009 TX-000005	**HOSPITAL_VERIFIED** H2 General Hospital is registered and active in the consortium registry.	Sara Malik Hospital Records Officer, H2	ACTIVE	94ac19cc22fc… ↓ **fa7d440dfadd…**
EVT-000010 TX-000004 REQ-2026-0001	**REQUEST_RECEIVED** H2 received REQ-2026-0001 in its inbox; Sara Malik acknowledged it.	Sara Malik Hospital Records Officer, H2	RECEIVED	fa7d440dfadd… ↓ **645980ce0623…**
EVT-000011 TX-000004 REQ-2026-0001	**REQUEST_REVIEW_STARTED** Sara Malik opened REQ-2026-0001 and started reviewing it.	Sara Malik Hospital Records Officer, H2	UNDER_REVIEW	645980ce0623… ↓ **85a2c1769e5a…**

Event / Tx / Request	Event type and explanation (as recorded)	Actor (role)	Result	previousHash → currentHash
EVT-000012 TX-000004 REQ-2026-0001	**HOSPITAL_VERIFICATION_COMPLETED** Sara Malik verified REQ-2026-0001: patient reference OK, requesting hospital OK, requesting professional OK, requested data held here OK, patient consent on file OK. This is the hospital's check; Algorithm 08 still decides independently.	Sara Malik Hospital Records Officer, H2	VERIFIED	85a2c1769e5a… ↓ **1c873ef1931d…**

Step 4 — Smart contract — Algorithm 08 applies the three rules

> **Assignment requirement:** Algorithm 08 executes; all three rules are evaluated individually, each showing its actual inputs, PASS/FAIL result and explanation.

What the user does: Clicks *Run Rule 1*, *Run Rule 2* and *Run Rule 3* (or *Run All Rules*).

What the system checks (logic): R1: both professionals authenticated and verified, H1 and H2 active, roles permitted → PASS. R2: consent CON-0001 grants H1, scope LAB and ECG includes ECG → PASS. R3: purpose Treatment = Treatment, ACTIVE, before 31 Dec 2026 → PASS. Result: CONTRACT_APPROVED.

Why: Each rule is recorded as its own event with its inputs, so the decision can be audited later.

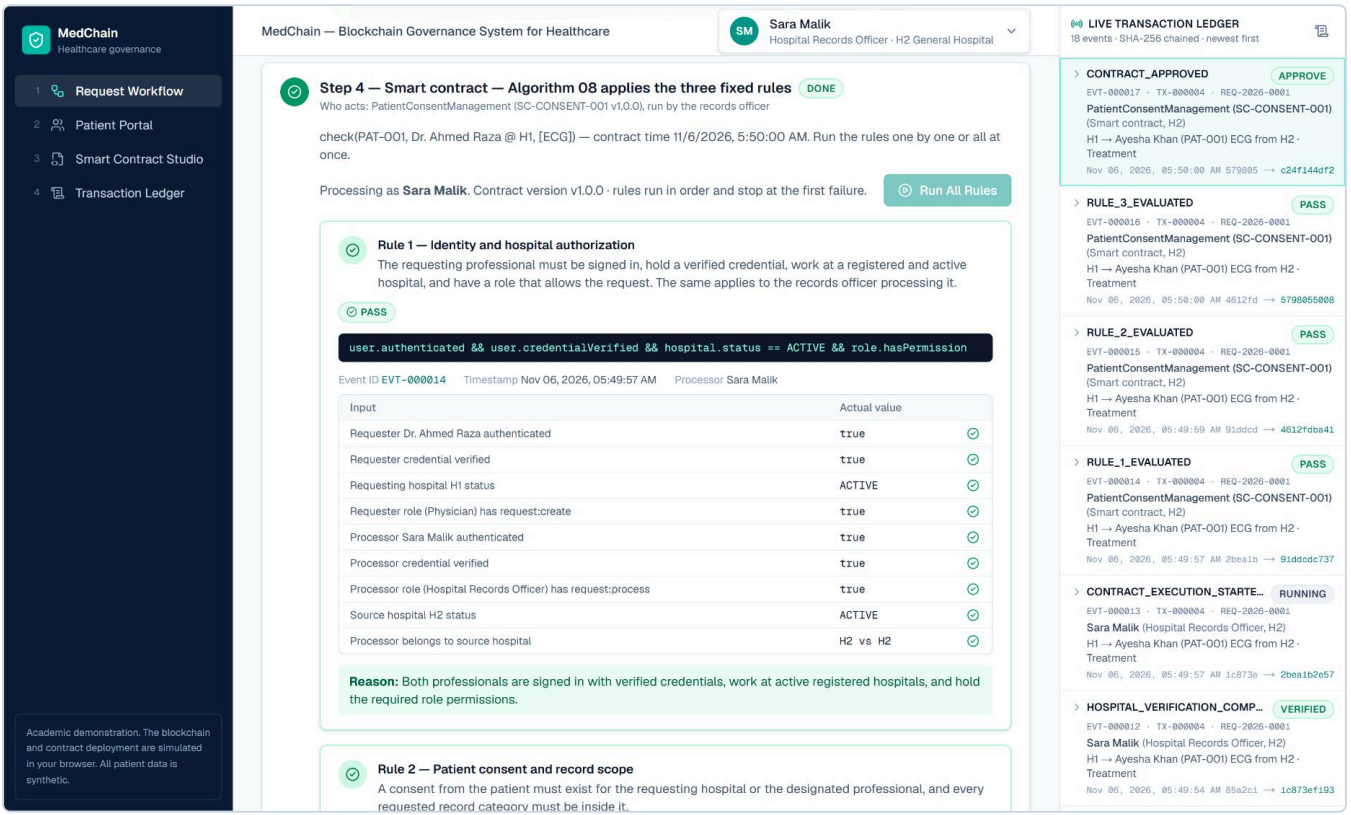


Figure 5. Rule 1 with every actual input and its result; the rule events appear in the live ledger.

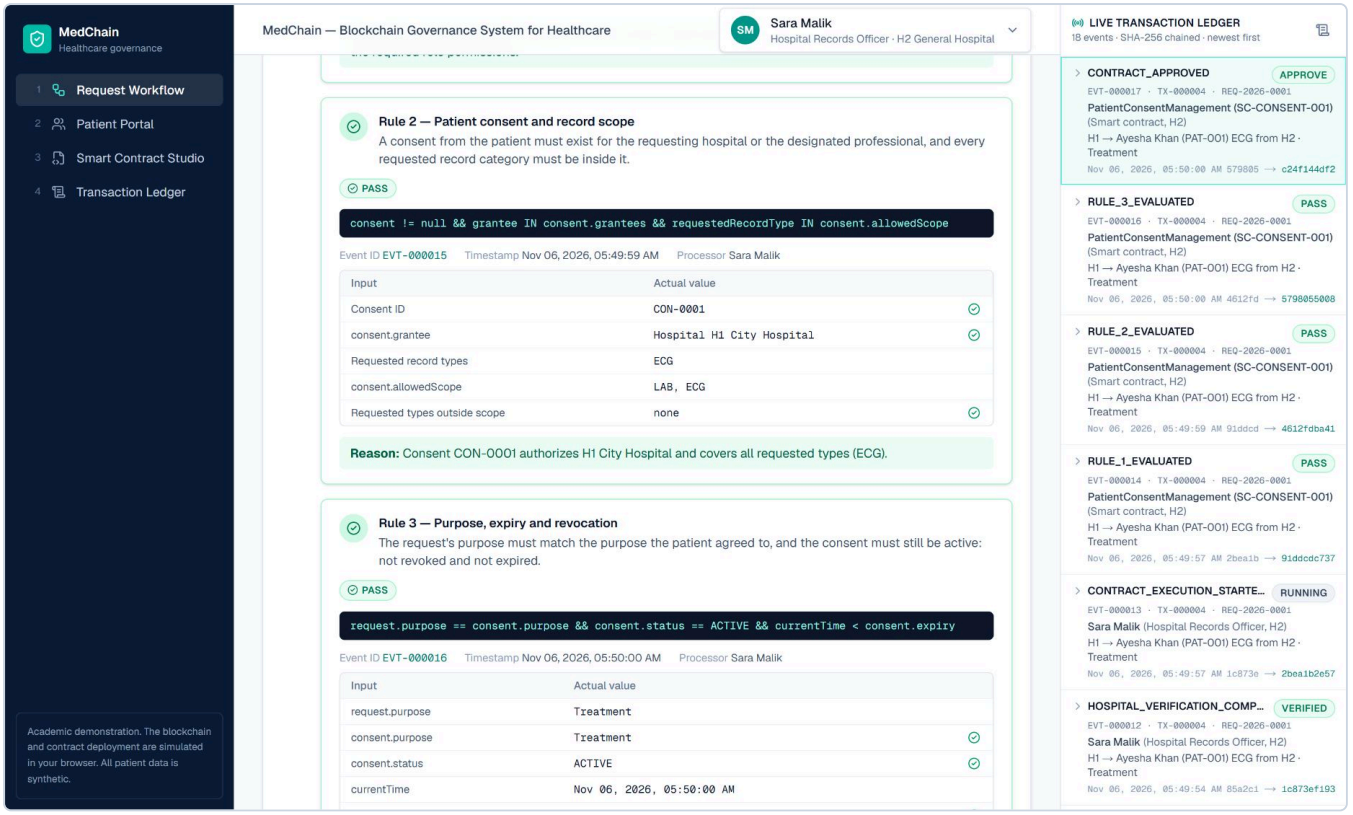


Figure 6. Rules 2 and 3 with their inputs; the contract approves (CONTRACT_APPROVED at the top of the live ledger).

Live ledger: events written by this step (5)

Event / Tx / Request	Event type and explanation (as recorded)	Actor (role)	Result	previousHash → currentHash
EVT-000013 TX-000004 REQ-2026-0001	**CONTRACT_EXECUTION_STARTED** Sara Malik started PatientConsentManagement v1.0.0: check(PAT-001, u-phy-h1@H1, [ECG]).	Sara Malik Hospital Records Officer, H2	RUNNING	1c873ef1931d… ↓ **2bea1b2e57cb…**
EVT-000014 TX-000004 REQ-2026-0001	**RULE_1_EVALUATED** Rule 1 (Identity and hospital authorization): Both professionals are signed in with verified credentials, work at active registered hospitals, and hold the required role permissions.	PatientConsentManagement (SC-CONSENT-001) Smart contract, H2	PASS	2bea1b2e57cb… ↓ **91ddcdc737ac…**
EVT-000015 TX-000004 REQ-2026-0001	**RULE_2_EVALUATED** Rule 2 (Patient consent and record scope): Consent CON-0001 authorizes H1 City Hospital and covers all requested types (ECG).	PatientConsentManagement (SC-CONSENT-001) Smart contract, H2	PASS	91ddcdc737ac… ↓ **4612fdba41c0…**
EVT-000016 TX-000004 REQ-2026-0001	**RULE_3_EVALUATED** Rule 3 (Purpose, expiry and revocation): Purpose matches and the consent is active, not revoked and not expired.	PatientConsentManagement (SC-CONSENT-001) Smart contract, H2	PASS	4612fdba41c0… ↓ **5798055008a6…**
EVT-000017 TX-000004 REQ-2026-0001	**CONTRACT_APPROVED** check() returned APPROVE: all three fixed rules passed. The source hospital must now approve or reject.	PatientConsentManagement (SC-CONSENT-001) Smart contract, H2	APPROVE	5798055008a6… ↓ **c24f144df2c5…**

Step 5 — Source hospital responds

> **Assignment requirement:** The Records Officer approves or rejects with a reason; the hospital cannot override patient consent or the rules.

What the user does: Clicks *Approve request*. (A *Reject with reason* button is also available.)

What the system checks (logic): Approval is accepted only when the request's status is CONTRACT_APPROVED, so a denied request can never be approved.

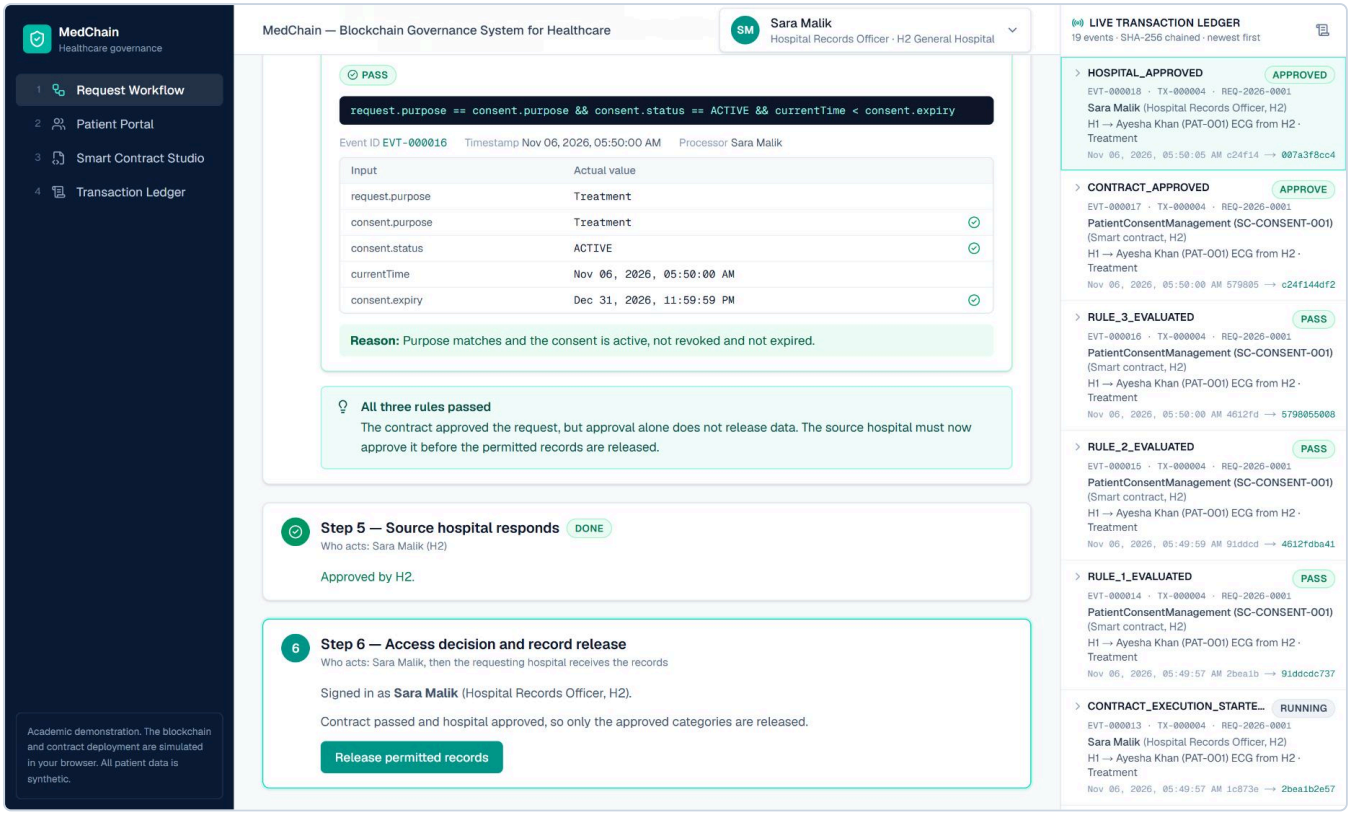


Figure 7. HOSPITAL_APPROVED recorded.

Live ledger: events written by this step (1)

Event / Tx / Request	Event type and explanation (as recorded)	Actor (role)	Result	previousHash → currentHash
EVT-000018 TX-000004 REQ-2026-0001	**HOSPITAL_APPROVED** Sara Malik (H2) approved REQ-2026-0001. Contract passed and hospital approved: the permitted records may now be released.	Sara Malik Hospital Records Officer, H2	APPROVED	c24f144df2c5… ↓ **007a3f8cc444…**

Step 6 — Access decision and medical record release

> **Assignment requirement:** If the hospital approves and all rules pass, the permitted medical history is released; the final decision and reason are displayed.

What the user does: Clicks *Release permitted records*.

What the system checks (logic): Released records = records of this patient, held by the source hospital, whose category was requested **and** is inside the consent. Here only REC-001-ECG is released; LAB (consented but not requested) and MRI (not consented) are not.

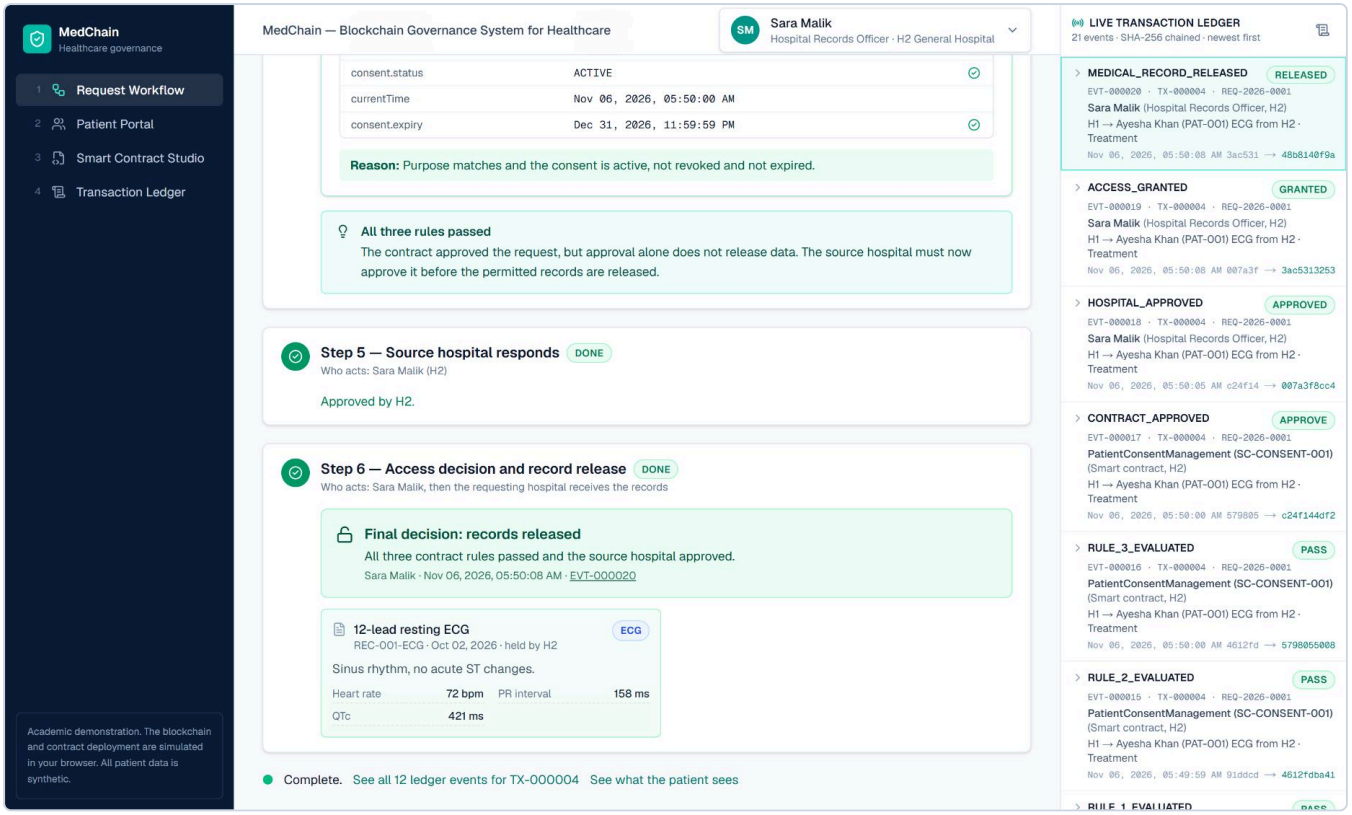


Figure 8. Final decision: records released. Only the ECG record is delivered.

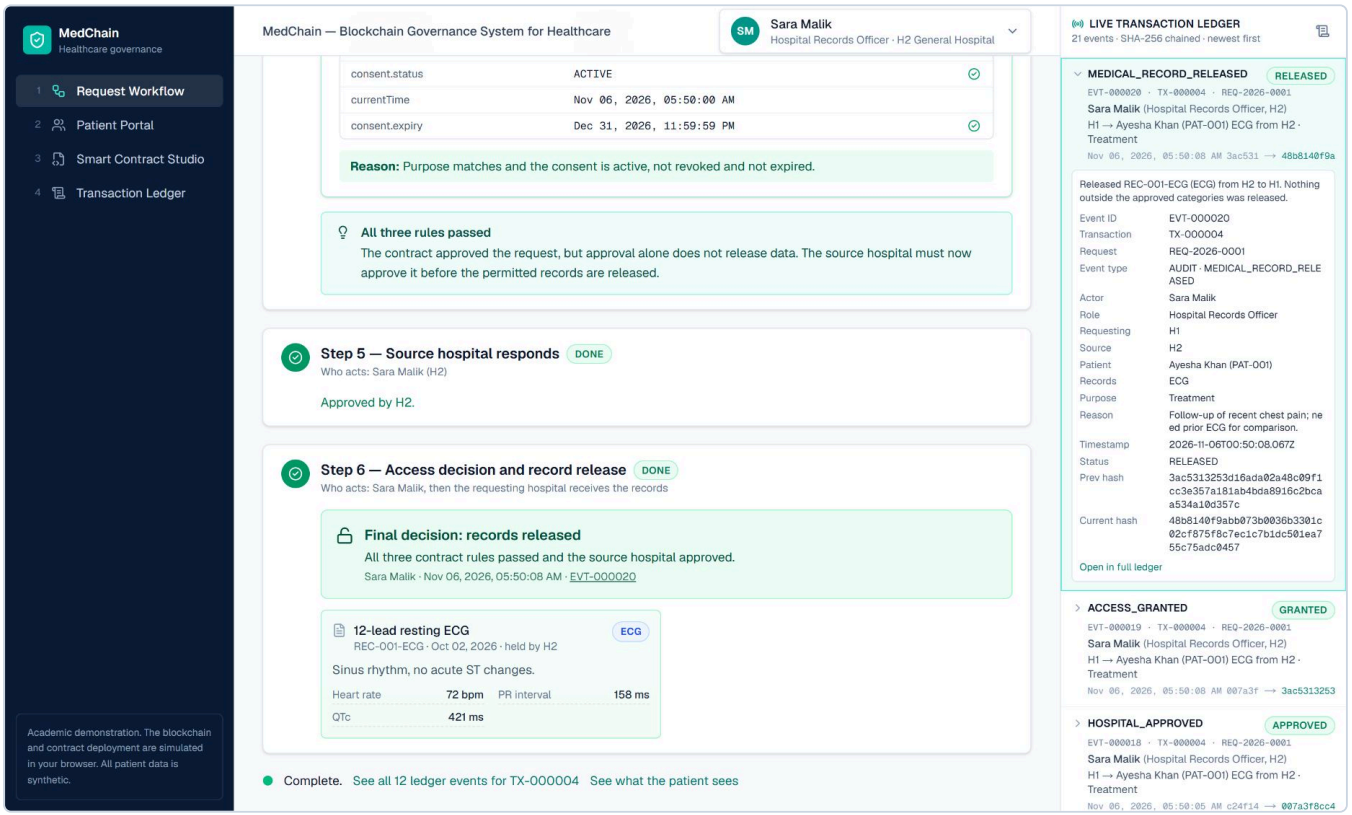


Figure 9. Live ledger with the MEDICAL_RECORD_RELEASED event expanded: event, transaction and request IDs, actor and role, requesting and source hospitals, patient, records, purpose, reason, timestamp and both full hashes.

Live ledger: events written by this step (2)

Event / Tx / Request	Event type and explanation (as recorded)	Actor (role)	Result	previousHash → currentHash
EVT-000019 TX-000004 REQ-2026-0001	**ACCESS_GRANTED** Access granted: all three contract rules passed and the source hospital approved.	Sara Malik Hospital Records Officer, H2	GRANTED	007a3f8cc444… ↓ **3ac5313253d1…**
EVT-000020 TX-000004 REQ-2026-0001	**MEDICAL_RECORD_RELEASED** Released REC-001-ECG (ECG) from H2 to H1. Nothing outside the approved categories was released.	Sara Malik Hospital Records Officer, H2	RELEASED	3ac5313253d1… ↓ **48b8140f9abb…**

7. Case 2 — MRI request outside consent (denied by Rule 2)

Same physician and patient, but the request asks for **MRI**, which the consent does not include. Request **REQ-2026-0002**, transaction **TX-000007**.

Request submitted

What the user does: Starts a new request, signs in as Dr. Ahmed Raza again, ticks MRI only, submits.

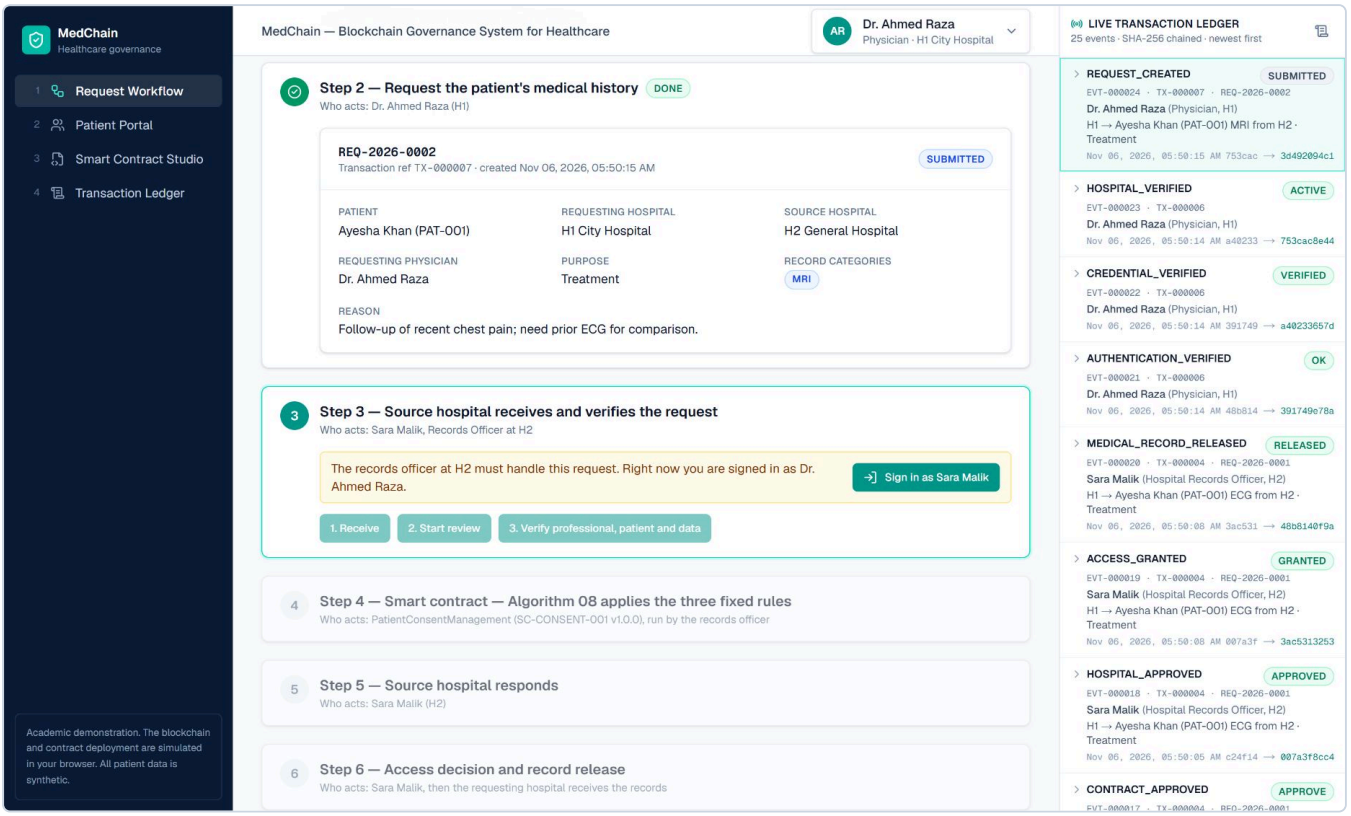


Figure 10. MRI request submitted; REQUEST_CREATED at the top of the live ledger.

Live ledger: events written by this step (4)

Event / Tx / Request	Event type and explanation (as recorded)	Actor (role)	Result	previousHash → currentHash
EVT-000021 TX-000006	**AUTHENTICATION_VERIFIED** Dr. Ahmed Raza signed in as Physician using DID did:medchain:user:bdbf7bc7b1f566b7 (demo authentication).	Dr. Ahmed Raza Physician, H1	OK	48b8140f9abb… ↓ **391749e78a21…**
EVT-000022 TX-000006	**CREDENTIAL_VERIFIED** Credential urn:vc:medchain:u-phy-h1 verified: trusted issuer, intact, in date, not revoked.	Dr. Ahmed Raza Physician, H1	VERIFIED	391749e78a21… ↓ **a40233657d6b…**
EVT-000023 TX-000006	**HOSPITAL_VERIFIED** H1 City Hospital is registered and active in the consortium registry.	Dr. Ahmed Raza Physician, H1	ACTIVE	a40233657d6b… ↓ **753cac8e44f6…**
EVT-000024 TX-000007 REQ-2026-0002	**REQUEST_CREATED** Dr. Ahmed Raza (H1) requested Ayesha Khan's MRI from H2 for Treatment. Reason: Follow-up of recent chest pain; need prior ECG for comparison.	Dr. Ahmed Raza Physician, H1	SUBMITTED	753cac8e44f6… ↓ **3d492094c1a3…**

Hospital verification and contract execution

What the system checks (logic): R1 passes (same authorized people). R2: requested MRI is not in the allowed scope (LAB, ECG) → FAIL. check() stops; R3 is not evaluated. CONTRACT_DENIED and ACCESS_DENIED are

recorded.

Why: The patient never consented to MRI sharing, so no hospital decision can release it.

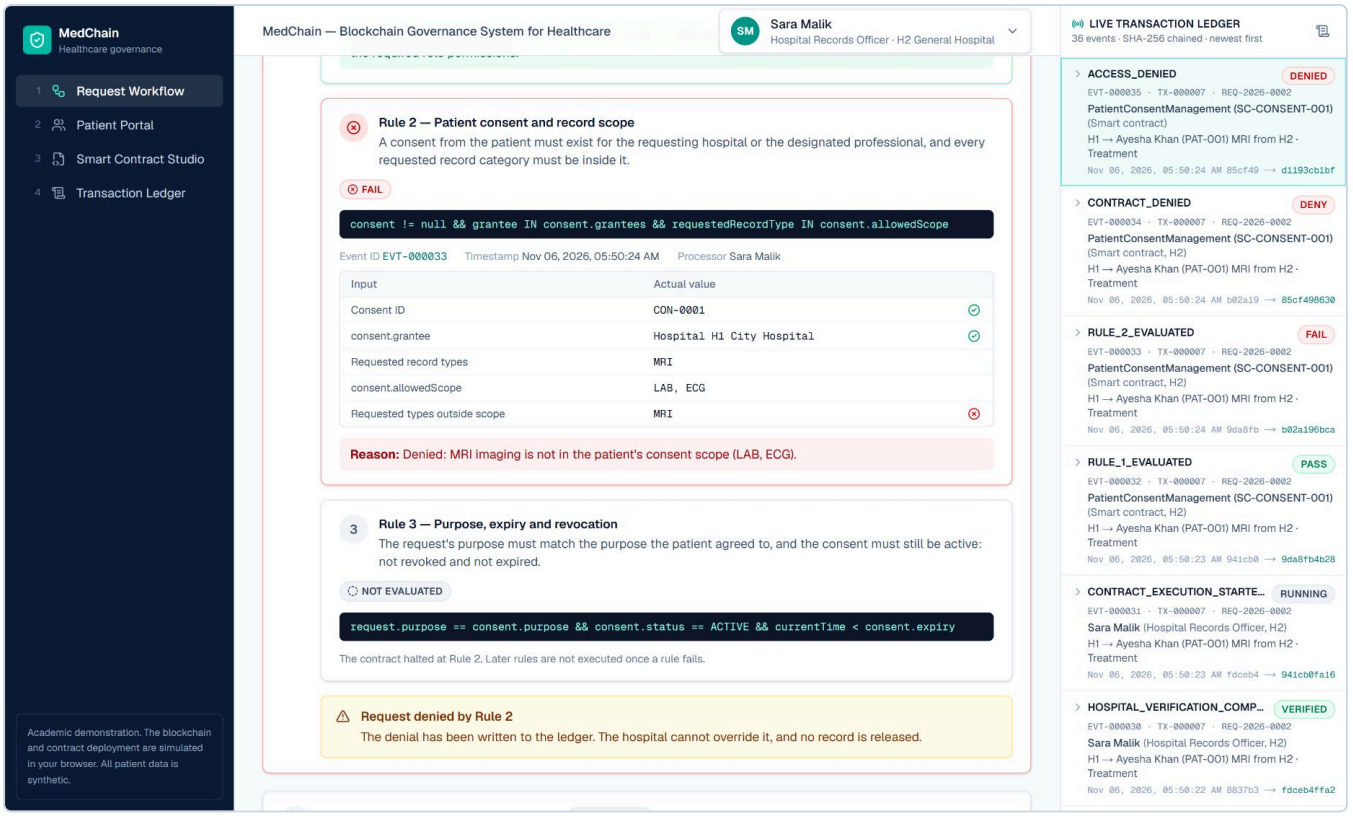


Figure 11. Rule 2 FAIL with its inputs: requested MRI, allowed LAB and ECG.

Live ledger: events written by this step (11)

Event / Tx / Request	Event type and explanation (as recorded)	Actor (role)	Result	previousHash → currentHash
EVT-000025 TX-000008	**AUTHENTICATION_VERIFIED** Sara Malik signed in as Hospital Records Officer using DID did:medchain:user:66ab9eea1309ceb0 (demo authentication).	Sara Malik Hospital Records Officer, H2	OK	3d492094c1a3… ↓ **9bfcd049327b…**
EVT-000026 TX-000008	**CREDENTIAL_VERIFIED** Credential urn:vc:medchain:u-rec-h2 verified: trusted issuer, intact, in date, not revoked.	Sara Malik Hospital Records Officer, H2	VERIFIED	9bfcd049327b… ↓ **2af01cb423a3…**
EVT-000027 TX-000008	**HOSPITAL_VERIFIED** H2 General Hospital is registered and active in the consortium registry.	Sara Malik Hospital Records Officer, H2	ACTIVE	2af01cb423a3… ↓ **8c4baee5e77d…**
EVT-000028 TX-000007 REQ-2026-0002	**REQUEST_RECEIVED** H2 received REQ-2026-0002 in its inbox; Sara Malik acknowledged it.	Sara Malik Hospital Records Officer, H2	RECEIVED	8c4baee5e77d… ↓ **edd966ee9b00…**
EVT-000029 TX-000007 REQ-2026-0002	**REQUEST_REVIEW_STARTED** Sara Malik opened REQ-2026-0002 and started reviewing it.	Sara Malik Hospital Records Officer, H2	UNDER_REVIEW	edd966ee9b00… ↓ **8837b351d717…**
EVT-000030 TX-000007 REQ-2026-0002	**HOSPITAL_VERIFICATION_COMPLETED** Sara Malik verified REQ-2026-0002: patient reference OK, requesting hospital OK, requesting professional OK, requested data held here OK, patient consent on file OK. This is the hospital's check; Algorithm 08 still decides independently.	Sara Malik Hospital Records Officer, H2	VERIFIED	8837b351d717… ↓ **fdceb4ffa23c…**

Event / Tx / Request	Event type and explanation (as recorded)	Actor (role)	Result	previousHash → currentHash
EVT-000031 TX-000007 REQ-2026-0002	**CONTRACT_EXECUTION_STARTED** Sara Malik started PatientConsentManagement v1.0.0: check(PAT-001, u-phy-h1@H1, [MRI]).	Sara Malik Hospital Records Officer, H2	RUNNING	fdceb4ffa23c… ↓ **941cb0fa16d1…**
EVT-000032 TX-000007 REQ-2026-0002	**RULE_1_EVALUATED** Rule 1 (Identity and hospital authorization): Both professionals are signed in with verified credentials, work at active registered hospitals, and hold the required role permissions.	PatientConsentManagement (SC-CONSENT-001) Smart contract, H2	PASS	941cb0fa16d1… ↓ **9da8fb4b28e3…**
EVT-000033 TX-000007 REQ-2026-0002	**RULE_2_EVALUATED** Rule 2 (Patient consent and record scope): Denied: MRI imaging is not in the patient's consent scope (LAB, ECG).	PatientConsentManagement (SC-CONSENT-001) Smart contract, H2	FAIL	9da8fb4b28e3… ↓ **b02a196bcaeb…**
EVT-000034 TX-000007 REQ-2026-0002	**CONTRACT_DENIED** check() returned DENY at Rule 2. A failed rule cannot be overridden.	PatientConsentManagement (SC-CONSENT-001) Smart contract, H2	DENY	b02a196bcaeb… ↓ **85cf49863085…**
EVT-000035 TX-000007 REQ-2026-0002	**ACCESS_DENIED** Access denied. Denied: MRI imaging is not in the patient's consent scope (LAB, ECG). No medical records were released.	PatientConsentManagement (SC-CONSENT-001) Smart contract	DENIED	85cf49863085… ↓ **d1193cb1bf10…**

Outcome

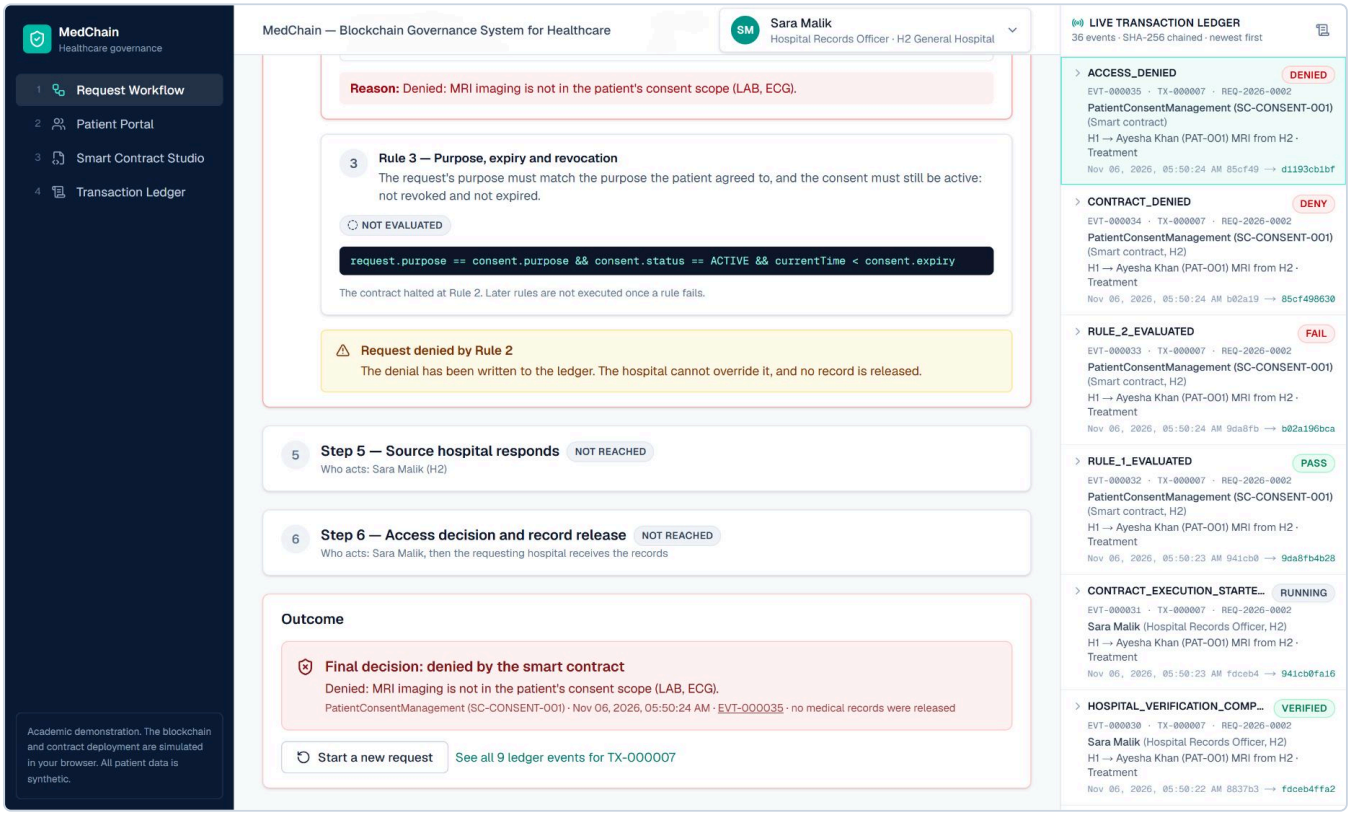


Figure 12. Steps 5 and 6 are 'Not reached'. No Approve or Release action exists. Final decision: denied by the smart contract; no medical records released.

8. Further denial cases

8.1 Unauthorized hospital — denied by Rule 1 (REQ-2026-0003)

Dr. Victor Hale works at H5 External Hospital, which is not registered. His credential is issued by H5, which is not a trusted consortium issuer.

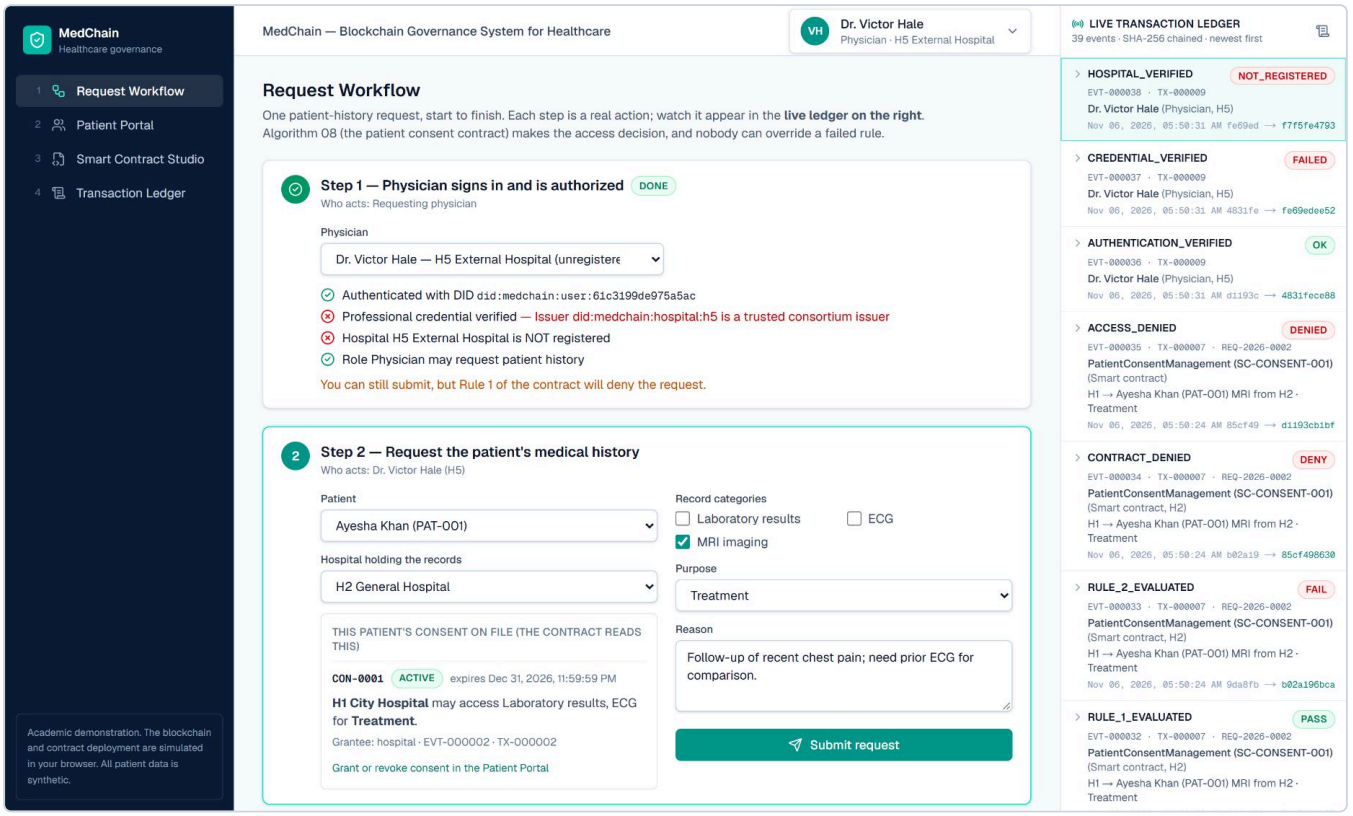


Figure 13. Sign-in: authenticated, but credential verification fails and the hospital is not registered. Submitting is allowed so that the contract makes the decision.

Events written at sign-in

Event / Tx / Request	Event type and explanation (as recorded)	Actor (role)	Result	previousHash → currentHash
EVT-000036 TX-000009	**AUTHENTICATION_VERIFIED** Dr. Victor Hale signed in as Physician using DID did:medchain:user:61c3199de975a5ac (demo authentication).	Dr. Victor Hale Physician, H5	OK	d1193cb1bf10… ↓ **4831fece8896…**
EVT-000037 TX-000009	**CREDENTIAL_VERIFIED** Credential verification failed: Issuer did:medchain:hospital:h5 is a trusted consortium issuer.	Dr. Victor Hale Physician, H5	FAILED	4831fece8896… ↓ **fe69edee52f4…**
EVT-000038 TX-000009	**HOSPITAL_VERIFIED** H5 External Hospital is not registered in the consortium registry.	Dr. Victor Hale Physician, H5	NOT_REGISTERED	fe69edee52f4… ↓ **f7f5fe4793ac…**

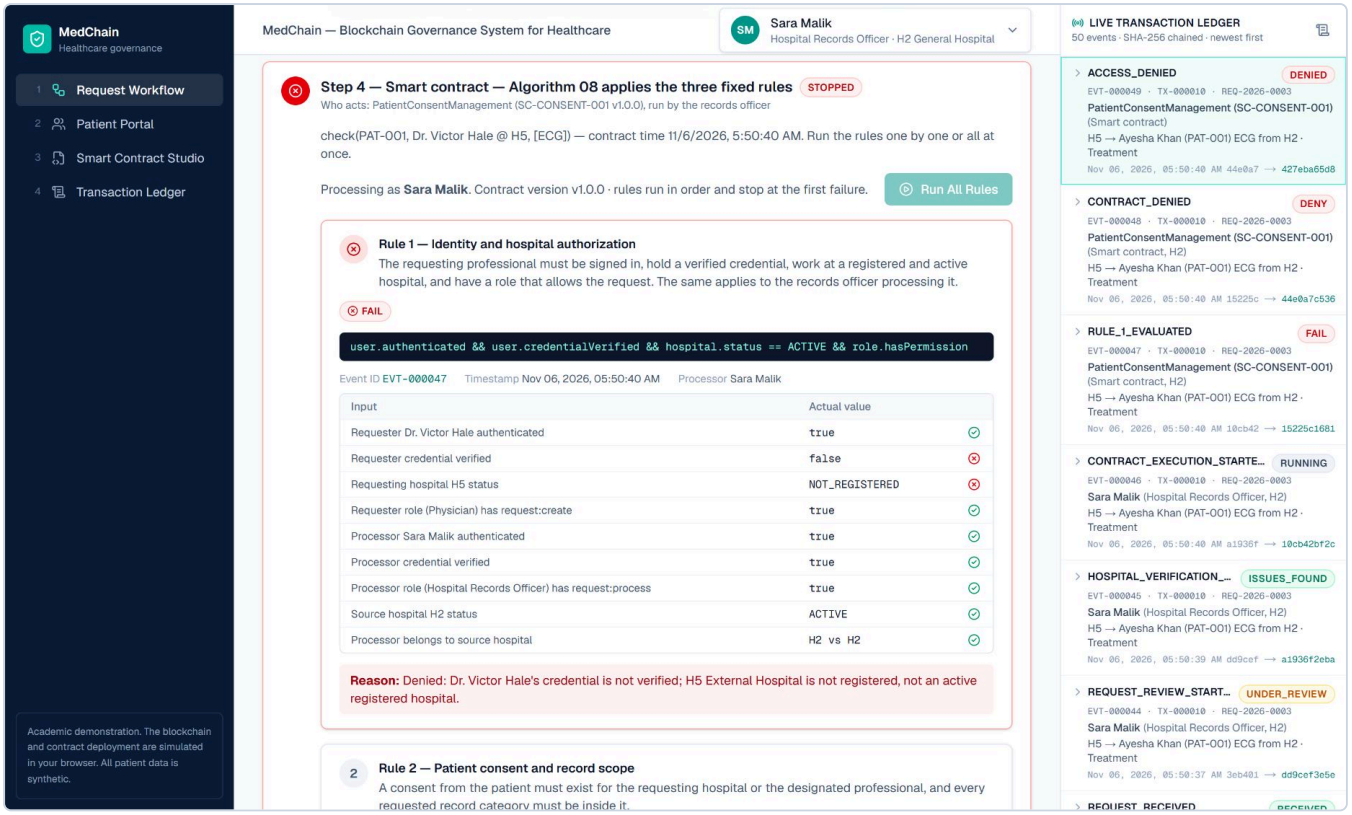


Figure 14. Rule 1 FAIL: requester credential not verified and H5 not ACTIVE. Rules 2 and 3 are not evaluated.

Events of this request

Event / Tx / Request	Event type and explanation (as recorded)	Actor (role)	Result	previousHash → currentHash
EVT-000039 TX-000010 REQ-2026-0003	**REQUEST_CREATED** Dr. Victor Hale (H5) requested Ayesha Khan's ECG from H2 for Treatment. Reason: Follow-up of recent chest pain; need prior ECG for comparison.	Dr. Victor Hale Physician, H5	SUBMITTED	f7f5fe4793ac… ↓ **bc4e2f018060…**
EVT-000043 TX-000010 REQ-2026-0003	**REQUEST_RECEIVED** H2 received REQ-2026-0003 in its inbox; Sara Malik acknowledged it.	Sara Malik Hospital Records Officer, H2	RECEIVED	ed8c6fe67bc9… ↓ **3eb401759843…**
EVT-000044 TX-000010 REQ-2026-0003	**REQUEST_REVIEW_STARTED** Sara Malik opened REQ-2026-0003 and started reviewing it.	Sara Malik Hospital Records Officer, H2	UNDER_REVIEW	3eb401759843… ↓ **dd9cef3e5e4a…**
EVT-000045 TX-000010 REQ-2026-0003	**HOSPITAL_VERIFICATION_COMPLETED** Sara Malik verified REQ-2026-0003: patient reference OK, requesting hospital flagged, requesting professional flagged, requested data held here OK, patient consent on file flagged. This is the hospital's check; Algorithm 08 still decides independently.	Sara Malik Hospital Records Officer, H2	ISSUES_FOUND	dd9cef3e5e4a… ↓ **a1936f2ebac2…**
EVT-000046 TX-000010 REQ-2026-0003	**CONTRACT_EXECUTION_STARTED** Sara Malik started PatientConsentManagement v1.0.0: check(PAT-001, u-phy-h5@H5, [ECG]).	Sara Malik Hospital Records Officer, H2	RUNNING	a1936f2ebac2… ↓ **10cb42bf2cd1…**
EVT-000047 TX-000010 REQ-2026-0003	**RULE_1_EVALUATED** Rule 1 (Identity and hospital authorization): Denied: Dr. Victor Hale's credential is not verified; H5 External Hospital is not registered, not an active registered hospital.	PatientConsentManagement (SC-CONSENT-001) Smart contract, H2	FAIL	10cb42bf2cd1… ↓ **15225c1681fc…**
EVT-000048 TX-000010 REQ-2026-0003	**CONTRACT_DENIED** check() returned DENY at Rule 1. A failed rule cannot be overridden.	PatientConsentManagement (SC-CONSENT-001) Smart contract, H2	DENY	15225c1681fc… ↓ **44e0a7c536a8…**

Event / Tx / Request	Event type and explanation (as recorded)	Actor (role)	Result	previousHash → currentHash
EVT-000049 TX-000010 REQ-2026-0003	**ACCESS_DENIED** Access denied. Denied: Dr. Victor Hale's credential is not verified; H5 External Hospital is not registered, not an active registered hospital. No medical records were released.	PatientConsentManagement (SC-CONSENT-001) Smart contract	DENIED	44e0a7c536a8... ↓ **427eba65d8fd...**

8.2 Revoked consent — denied by Rule 3 (REQ-2026-0004)

Ayesha revokes City Hospital's permission in the Patient Portal (`revoke(PAT-001, H1)`). Dr. Ahmed Raza then requests the ECG again.

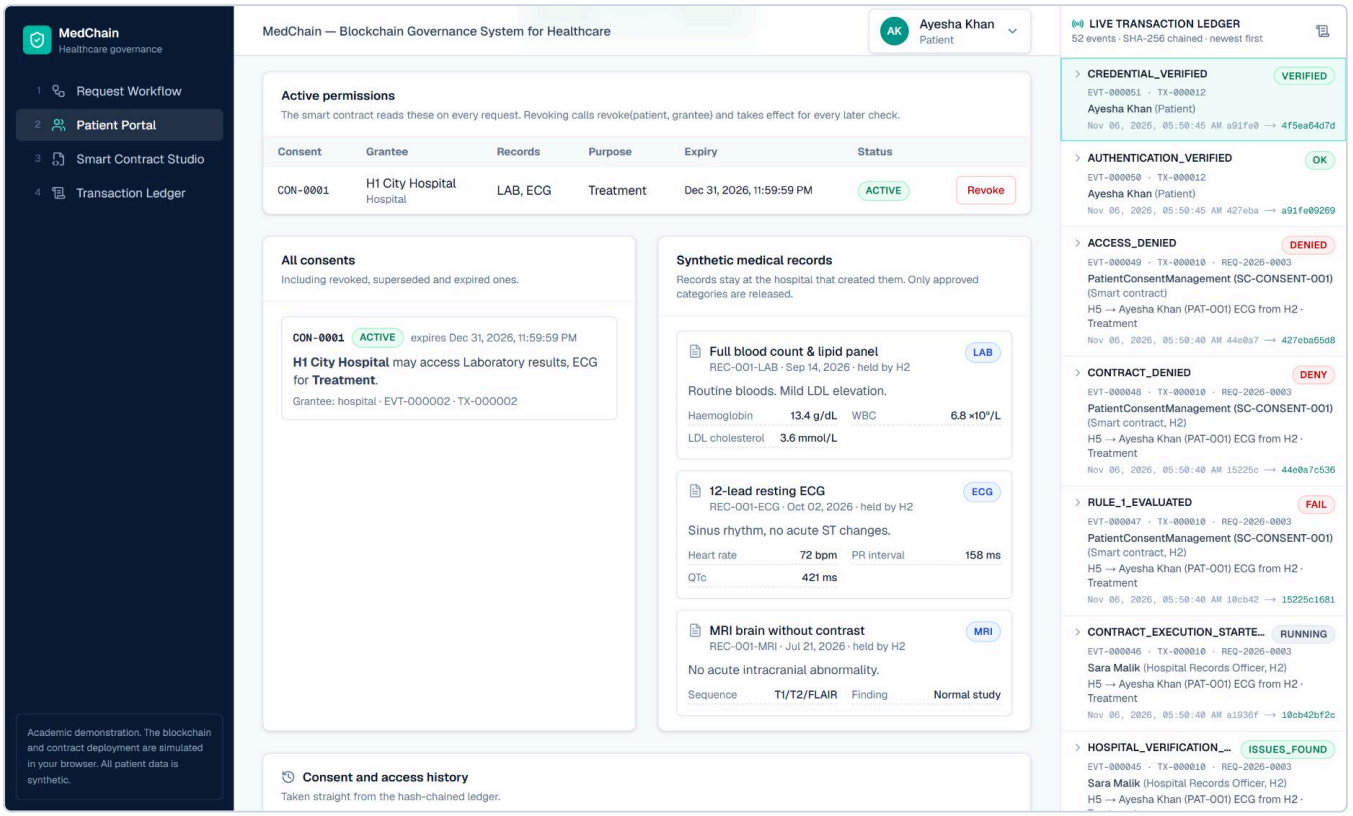


Figure 15. Before: City Hospital may access LAB and ECG for Treatment until 31 Dec 2026.

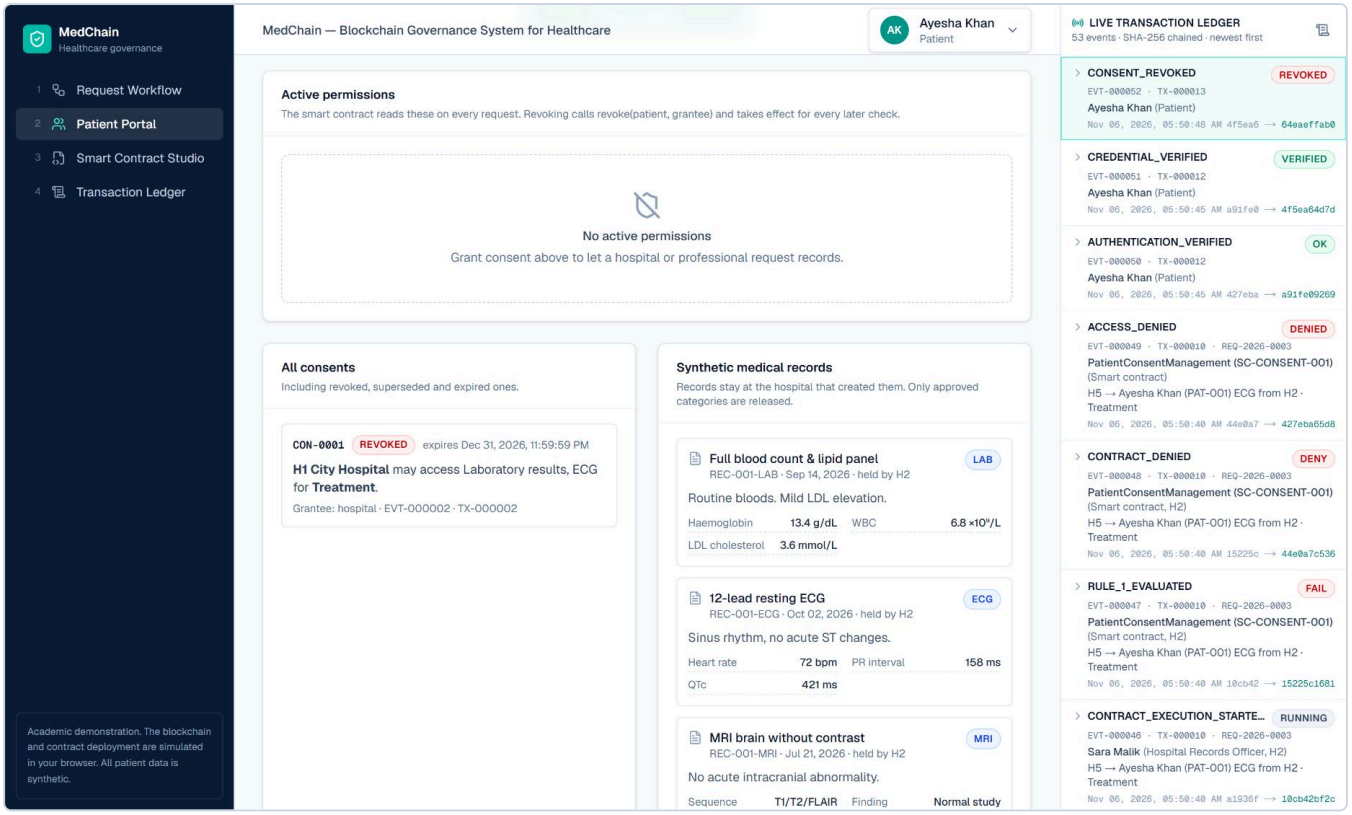


Figure 16. After revoke(): no active permissions; CONSENT_REVOKED at the top of the live ledger.

Revocation event

Event / Tx / Request	Event type and explanation (as recorded)	Actor (role)	Result	previousHash → currentHash
EVT-000052 TX-000013	**CONSENT_REVOKED** revoke(PAT-001, H1): Ayesha Khan withdrew permission for H1 City Hospital. Later contract checks will fail Rule 3.	Ayesha Khan Patient	REVOKED	4f5ea64d7d4c… ↓ **64eaeffab0d4…**

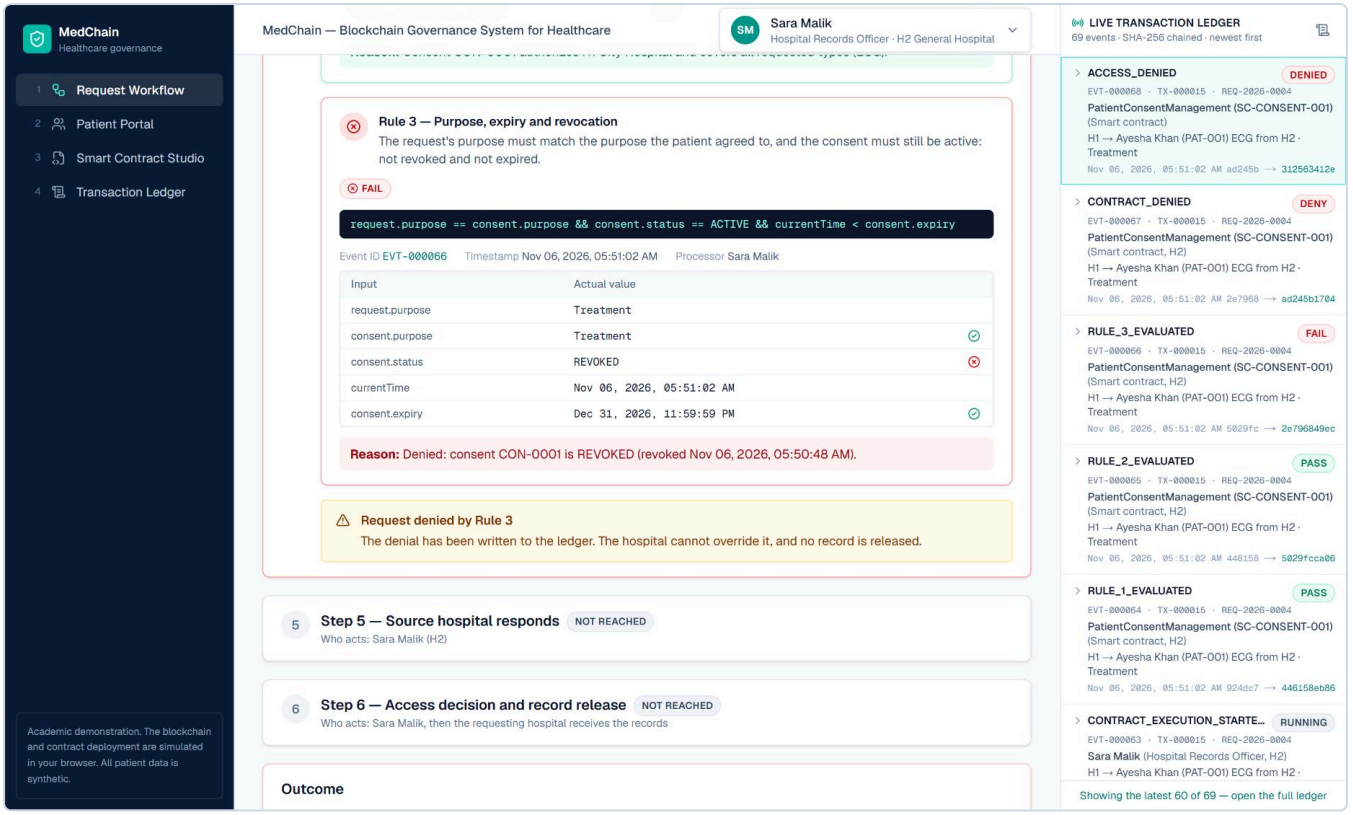


Figure 17. R1 and R2 pass (the revoked consent still names H1 and ECG), R3 fails: consent.status = REVOKED.

Contract and access events of this request

Event / Tx / Request	Event type and explanation (as recorded)	Actor (role)	Result	previousHash → currentHash
EVT-000063 TX-000015 REQ-2026-0004	**CONTRACT_EXECUTION_STARTED** Sara Malik started PatientConsentManagement v1.0.0: check(PAT-001, u-phy-h1@H1, [ECG]).	Sara Malik Hospital Records Officer, H2	RUNNING	9800ec4a603e… ↓ **924dc72426ef…**
EVT-000064 TX-000015 REQ-2026-0004	**RULE_1_EVALUATED** Rule 1 (Identity and hospital authorization): Both professionals are signed in with verified credentials, work at active registered hospitals, and hold the required role permissions.	PatientConsentManagement (SC-CONSENT-001) Smart contract, H2	PASS	924dc72426ef… ↓ **446158eb86ff…**
EVT-000065 TX-000015 REQ-2026-0004	**RULE_2_EVALUATED** Rule 2 (Patient consent and record scope): Consent CON-0001 authorizes H1 City Hospital and covers all requested types (ECG).	PatientConsentManagement (SC-CONSENT-001) Smart contract, H2	PASS	446158eb86ff… ↓ **5029fcca067d…**
EVT-000066 TX-000015 REQ-2026-0004	**RULE_3_EVALUATED** Rule 3 (Purpose, expiry and revocation): Denied: consent CON-0001 is REVOKED (revoked Nov 06, 2026, 05:50:48 AM).	PatientConsentManagement (SC-CONSENT-001) Smart contract, H2	FAIL	5029fcca067d… ↓ **2e796849ece7…**
EVT-000067 TX-000015 REQ-2026-0004	**CONTRACT_DENIED** check() returned DENY at Rule 3. A failed rule cannot be overridden.	PatientConsentManagement (SC-CONSENT-001) Smart contract, H2	DENY	2e796849ece7… ↓ **ad245b170428…**
EVT-000068 TX-000015 REQ-2026-0004	**ACCESS_DENIED** Access denied. Denied: consent CON-0001 is REVOKED (revoked Nov 06, 2026, 05:50:48 AM). No medical records were released.	PatientConsentManagement (SC-CONSENT-001) Smart contract	DENIED	ad245b170428… ↓ **312563412e34…**

Reasoning: revocation takes effect on the very next check because check() reads the current consent status at execution time; nothing is cached.

8.3 Expired consent — denied by Rule 3 (REQ-2026-0005)

Ayesha grants a new 7-day consent to City Hospital. The demo clock (used only for contract time) is then moved 14 days ahead to simulate the consent running out. Ledger timestamps stay real.

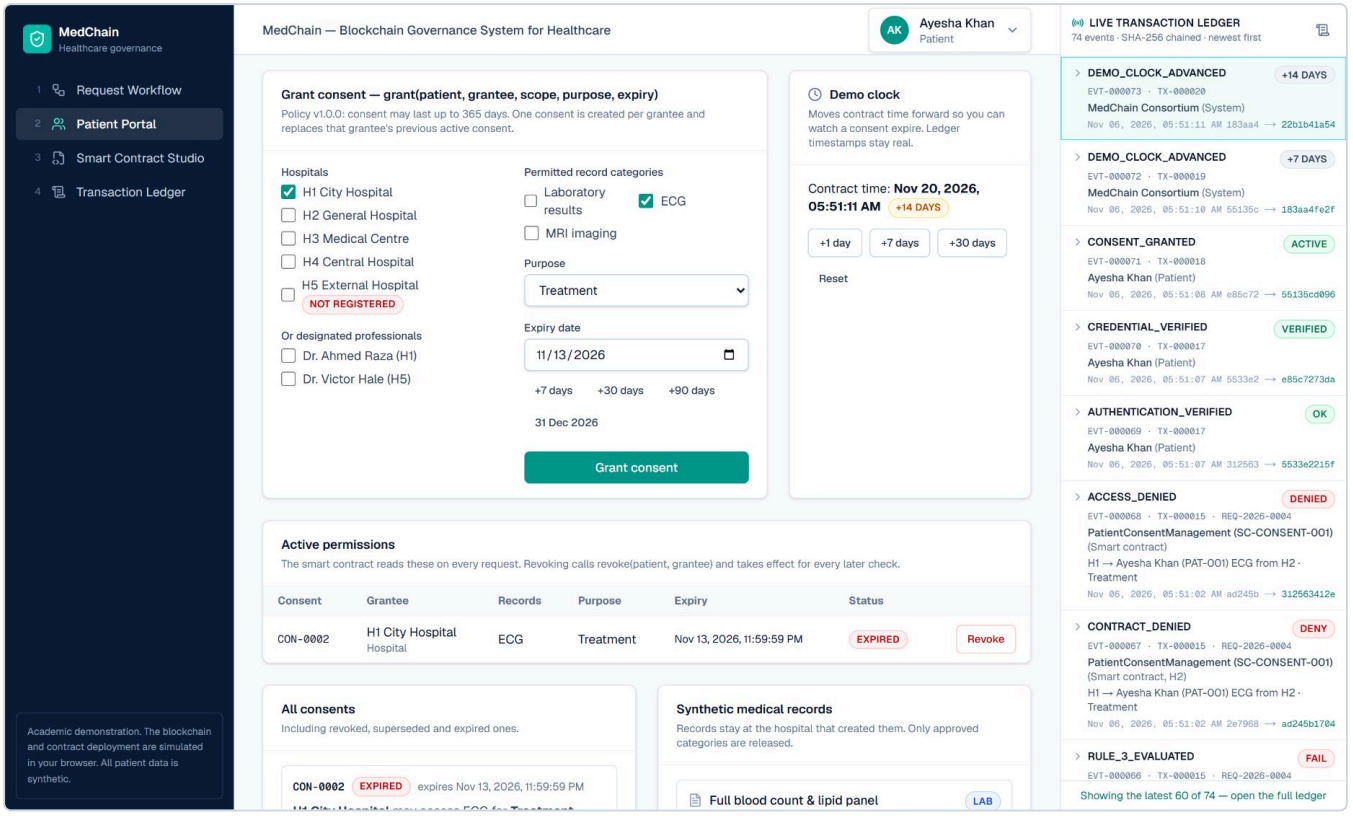


Figure 18. New 7-day consent granted; the demo clock is advanced. Each action is recorded in the live ledger.

Consent and clock events

Event / Tx / Request	Event type and explanation (as recorded)	Actor (role)	Result	previousHash → currentHash
EVT-000071 TX-000018	**CONSENT_GRANTED** grant(PAT-001, H1, [ECG], TREATMENT, 2026-11-13): Ayesha Khan allowed H1 City Hospital to access ECG for Treatment.	Ayesha Khan Patient	ACTIVE	e85c7273da70… ↓ **55135cd096c9…**
EVT-000072 TX-000019	**DEMO_CLOCK_ADVANCED** Demo clock moved forward 7 day(s). Contract time is now Nov 13, 2026, 05:51:10 AM. Event timestamps stay real.	MedChain Consortium System	+7 days	55135cd096c9… ↓ **183aa4fe2fa7…**
EVT-000073 TX-000020	**DEMO_CLOCK_ADVANCED** Demo clock moved forward 7 day(s). Contract time is now Nov 20, 2026, 05:51:11 AM. Event timestamps stay real.	MedChain Consortium System	+14 days	183aa4fe2fa7… ↓ **22b1b41a54c1…**

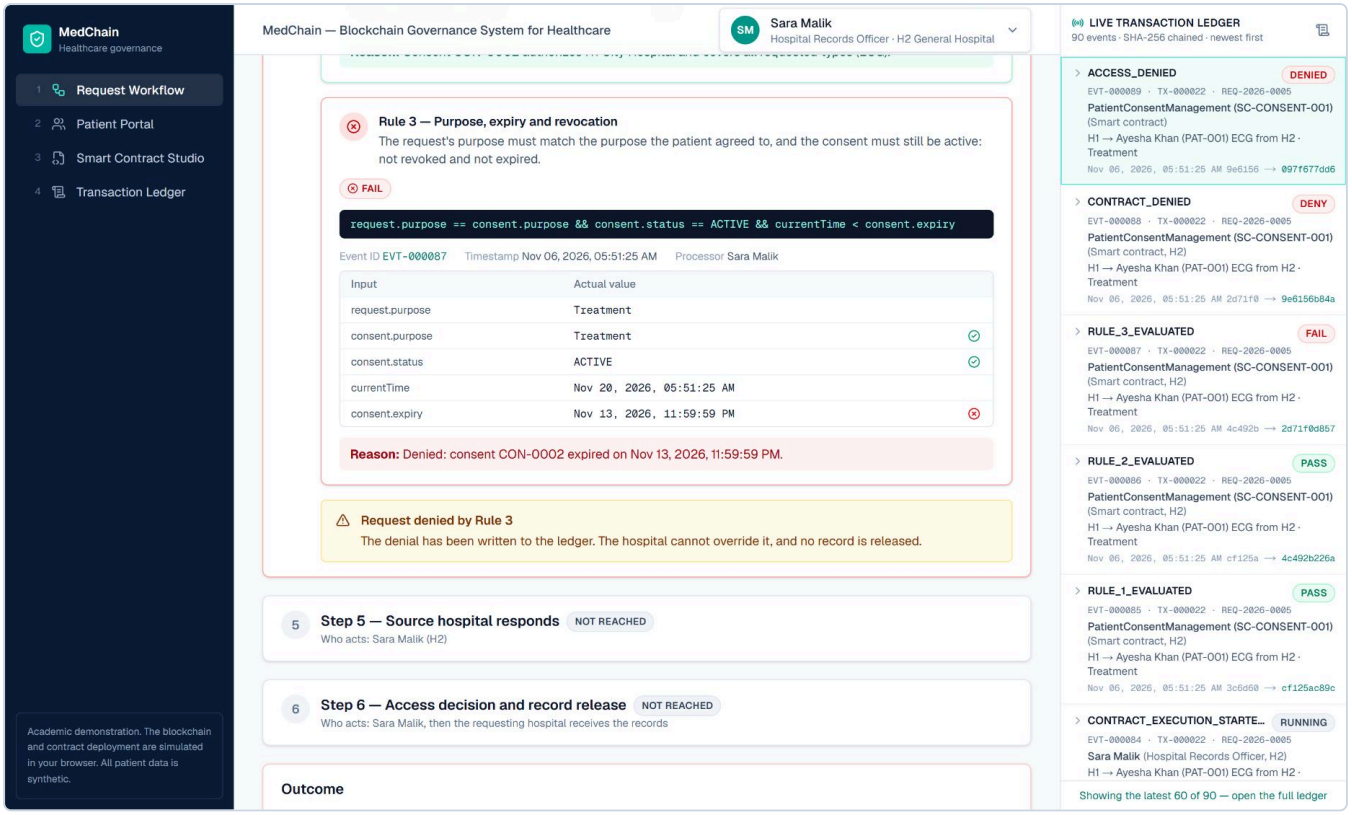


Figure 19. R3 FAIL: currentTime is after consent.expiry.

Contract and access events of this request

Event / Tx / Request	Event type and explanation (as recorded)	Actor (role)	Result	previousHash → currentHash
EVT-000084 TX-000022 REQ-2026-0005	**CONTRACT_EXECUTION_STARTED** Sara Malik started PatientConsentManagement v1.0.0: check(PAT-001, u-phy-h1@H1, [ECG]).	Sara Malik Hospital Records Officer, H2	RUNNING	1cb980a68119… ↓ **3c6d601e77aa…**
EVT-000085 TX-000022 REQ-2026-0005	**RULE_1_EVALUATED** Rule 1 (Identity and hospital authorization): Both professionals are signed in with verified credentials, work at active registered hospitals, and hold the required role permissions.	PatientConsentManagement (SC-CONSENT-001) Smart contract, H2	PASS	3c6d601e77aa… ↓ **cf125ac89c53…**
EVT-000086 TX-000022 REQ-2026-0005	**RULE_2_EVALUATED** Rule 2 (Patient consent and record scope): Consent CON-0002 authorizes H1 City Hospital and covers all requested types (ECG).	PatientConsentManagement (SC-CONSENT-001) Smart contract, H2	PASS	cf125ac89c53… ↓ **4c492b226a98…**
EVT-000087 TX-000022 REQ-2026-0005	**RULE_3_EVALUATED** Rule 3 (Purpose, expiry and revocation): Denied: consent CON-0002 expired on Nov 13, 2026, 11:59:59 PM.	PatientConsentManagement (SC-CONSENT-001) Smart contract, H2	FAIL	4c492b226a98… ↓ **2d71f0d85782…**
EVT-000088 TX-000022 REQ-2026-0005	**CONTRACT_DENIED** check() returned DENY at Rule 3. A failed rule cannot be overridden.	PatientConsentManagement (SC-CONSENT-001) Smart contract, H2	DENY	2d71f0d85782… ↓ **9e6156b84aa2…**
EVT-000089 TX-000022 REQ-2026-0005	**ACCESS_DENIED** Access denied. Denied: consent CON-0002 expired on Nov 13, 2026, 11:59:59 PM. No medical records were released.	PatientConsentManagement (SC-CONSENT-001) Smart contract	DENIED	9e6156b84aa2… ↓ **097f677dd6c3…**

9. Patient Portal — consent and access history

The patient can see their profile, record categories and who may currently see each one; grant consent (to hospitals or designated professionals; purpose Treatment or Research; expiry); see active permissions and revoke them; and see every request for their records with the requesting hospital, the decision and the reason.

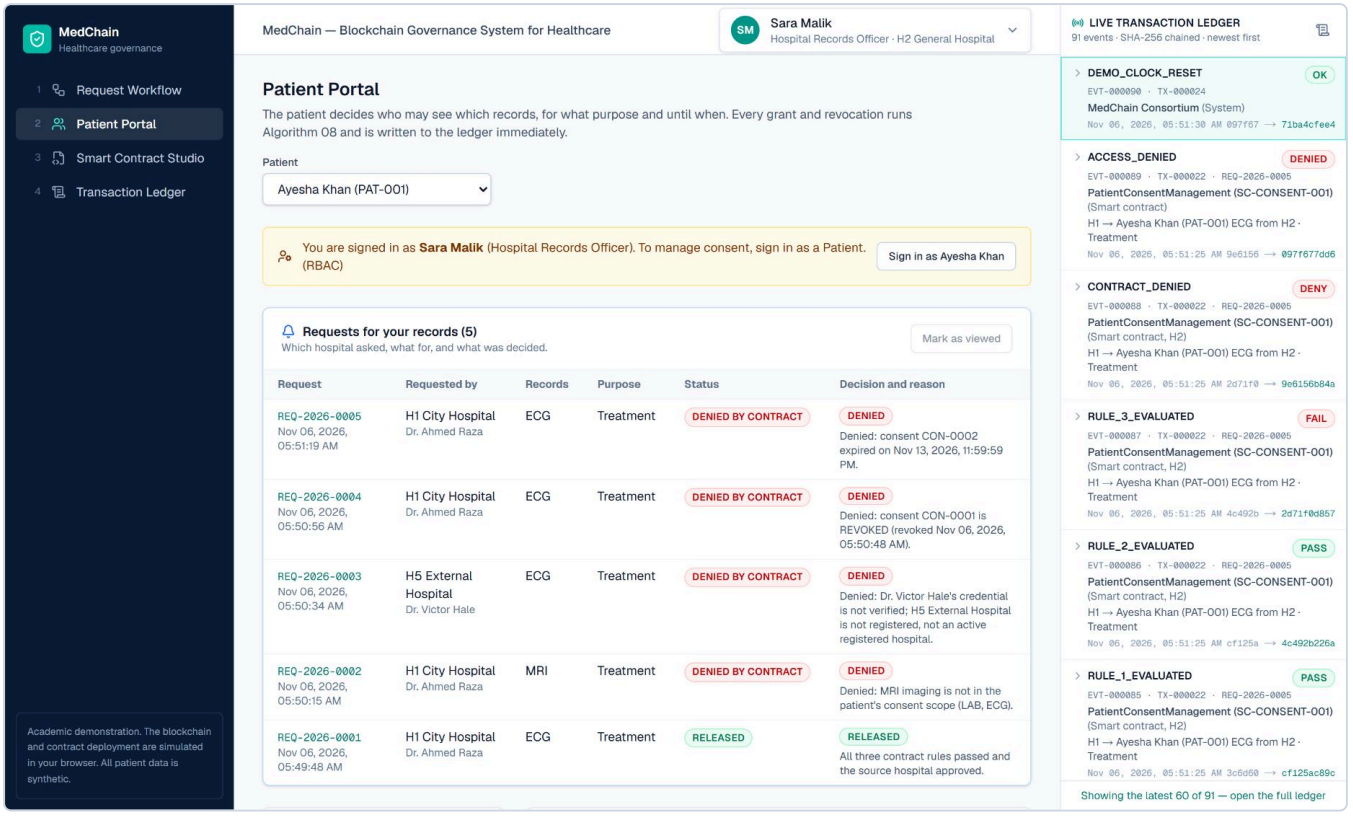


Figure 20. Patient Portal: all requests for Ayesha's records with status, decision and reason.

10. Ledger integrity — verification, tampering and hash inspection

Verification logic: for every event in order, check that its previousHash equals the stored hash of the event before it, then recompute SHA256(previousHash + canonicalEventPayload) and compare it with the stored currentHash. The first mismatch is reported with its position.

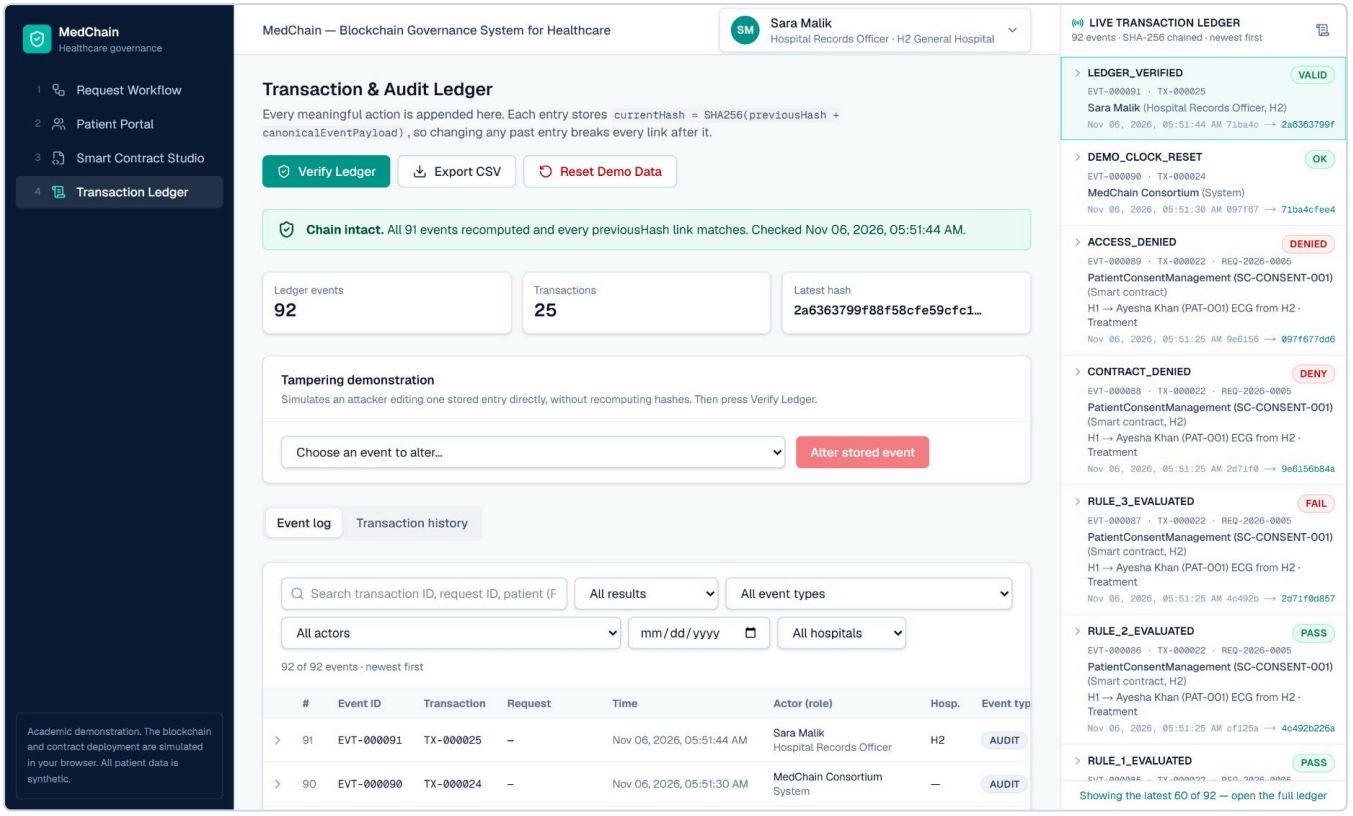


Figure 21. Verify Ledger: chain intact; every event recomputed.

Tampering demonstration: the tool edits one stored event directly, the way an attacker editing storage would, without recomputing any hash and without writing a ledger event.

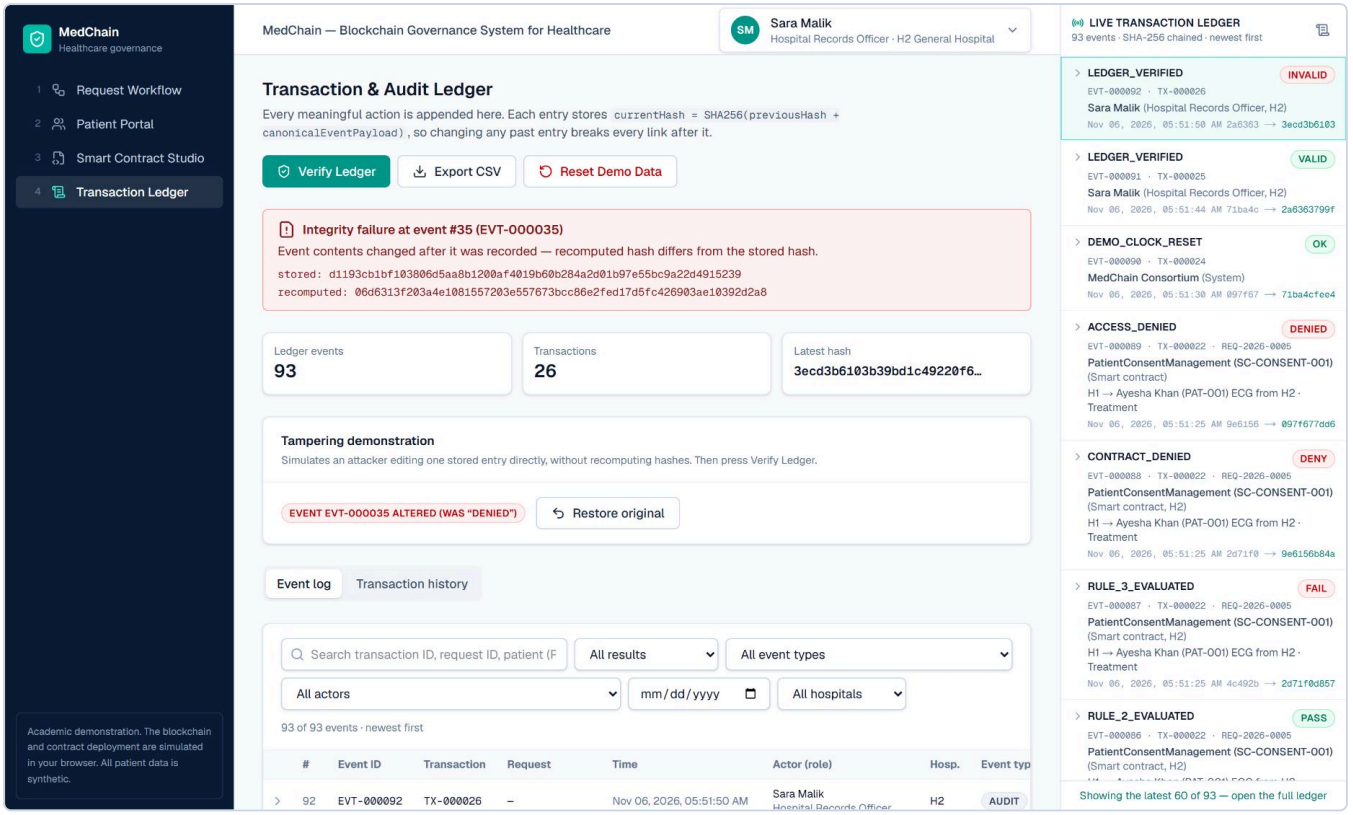


Figure 22. An ACCESS_DENIED event was changed to GRANTED in storage. Verification fails at exactly that event and shows the stored hash next to the recomputed one.

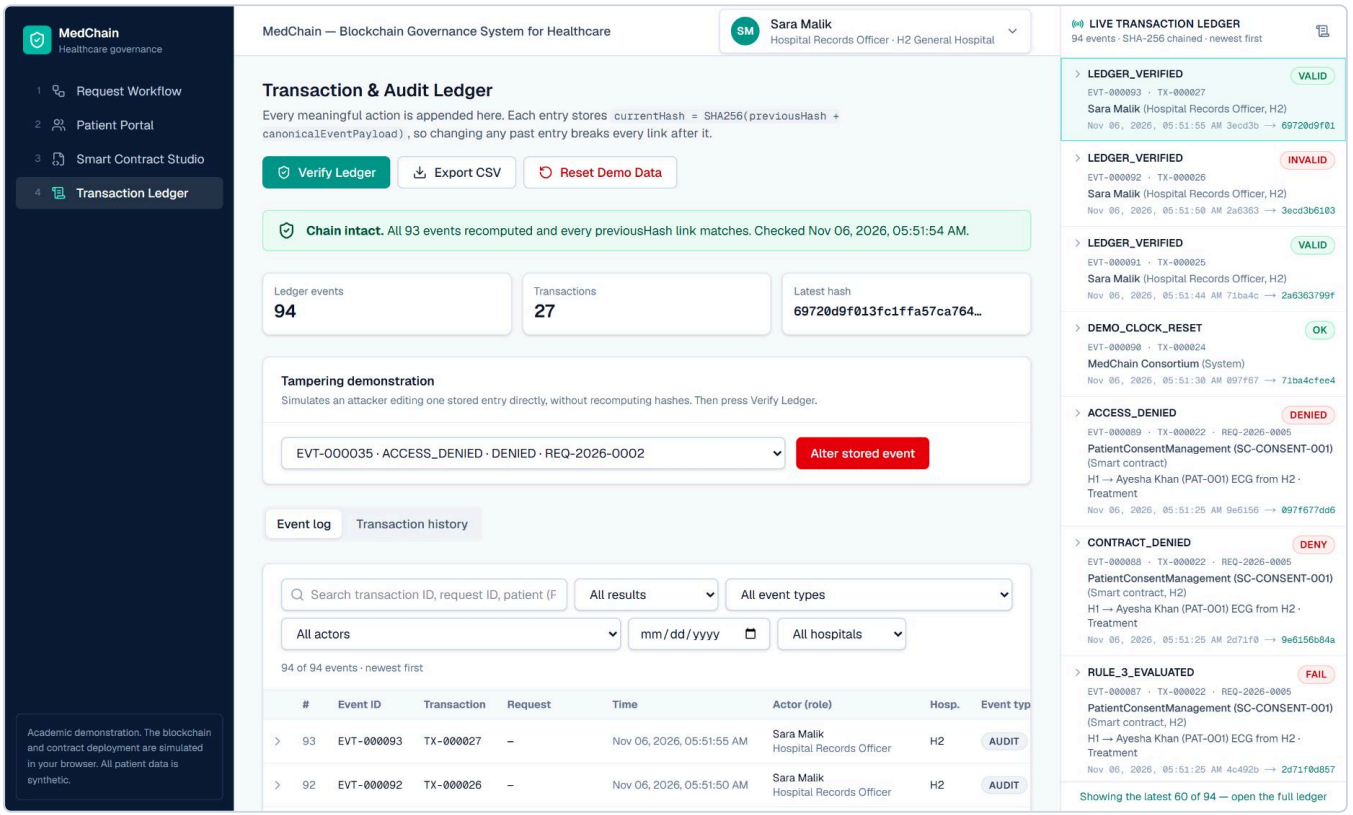


Figure 23. After 'Restore original', verification passes again.

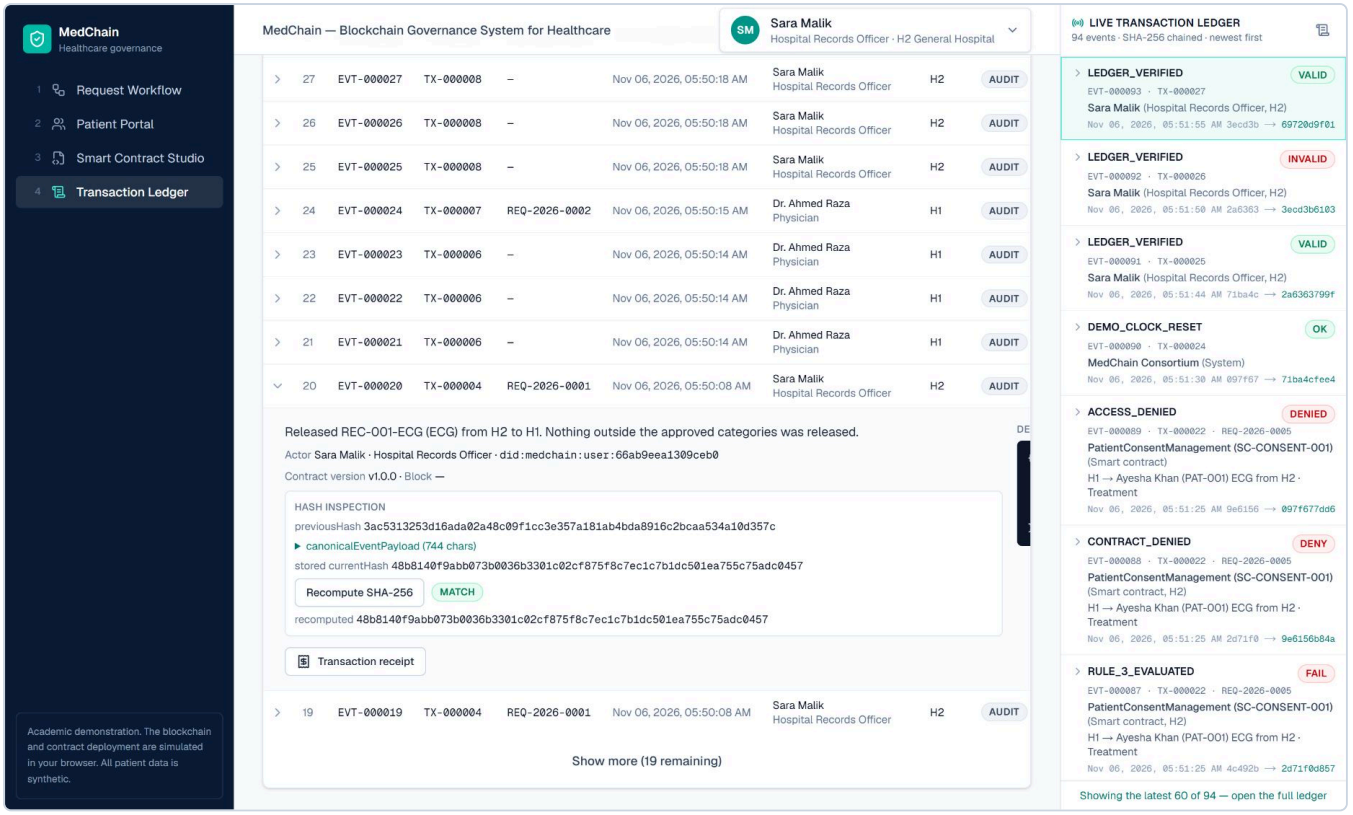


Figure 24. Hash inspection for one event: previousHash, the canonical payload, the stored hash and the hash recomputed live with SHA-256 — they match.

The ledger screen also provides search by transaction ID, request ID or patient; filters by event type, actor, hospital, date and outcome; a transaction-history view grouping events by transaction ID; a transaction receipt per event; and CSV export of all events with every field.

11. Smart Contract Studio

Contract: PatientConsentManagement — ID **SC-CONSENT-001**, version **1.0.0**, SHA-256 code hash `d51a7861d350c1e5fde782f5850cf354315b353938a6132d9ba8bdf23dc28f8b`. Deployment is simulated and produces a receipt (transaction, block, deployer, simulated address).

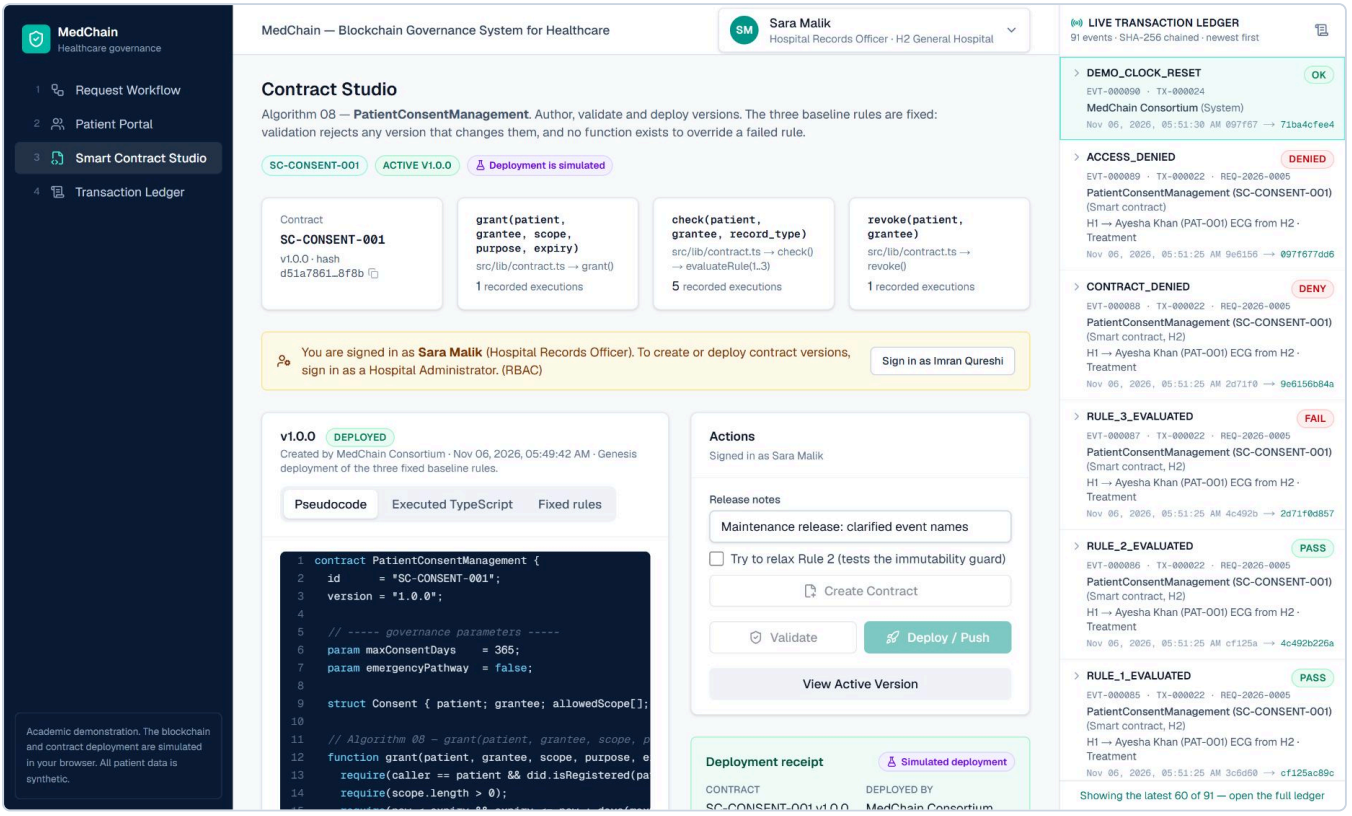


Figure 25. Contract ID, active version and hash; pseudocode with grant(), check(), revoke(); the shaded lines are the fixed rules.

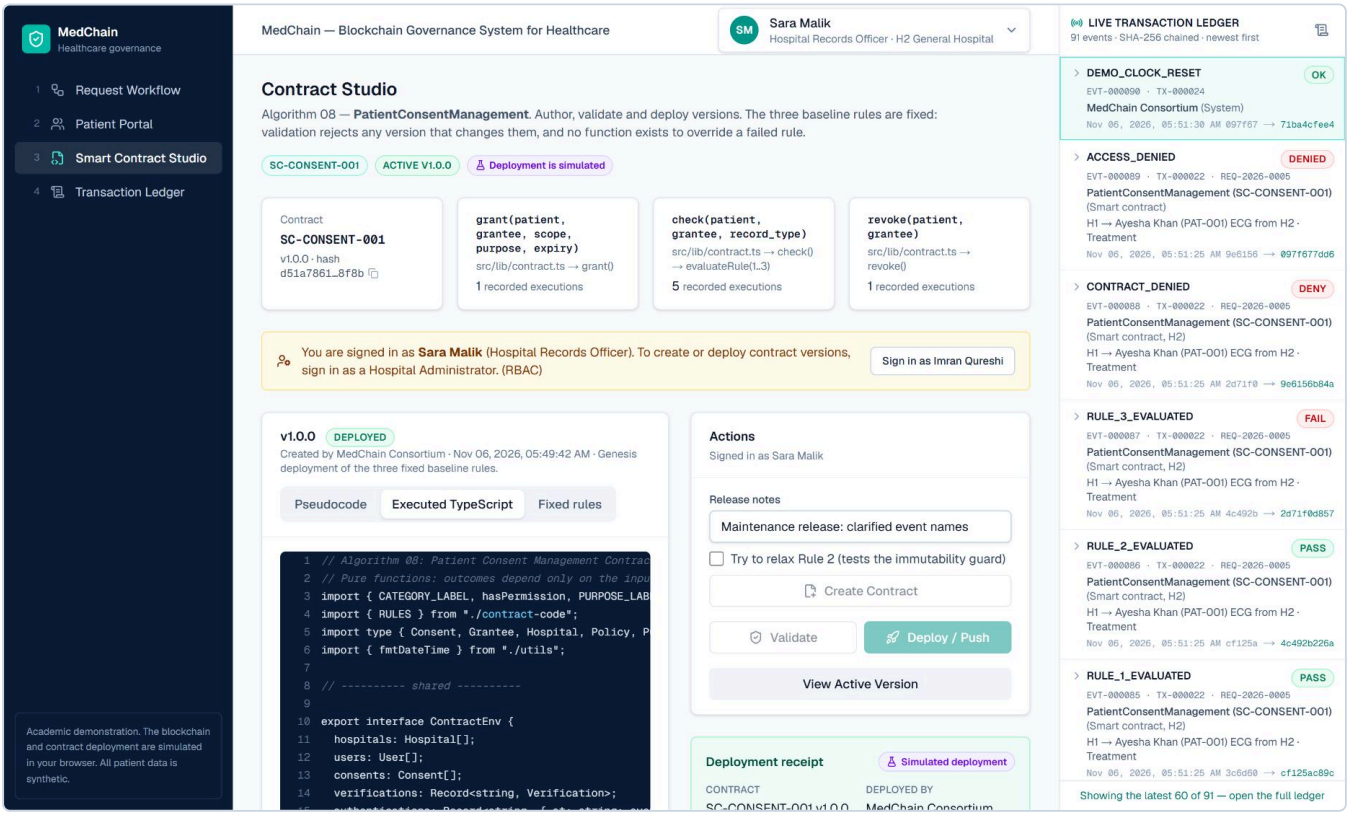


Figure 26. The TypeScript the application actually executes (src/lib/contract.ts), served from the source file, with its SHA-256 hash, so the displayed algorithm matches the executed code.

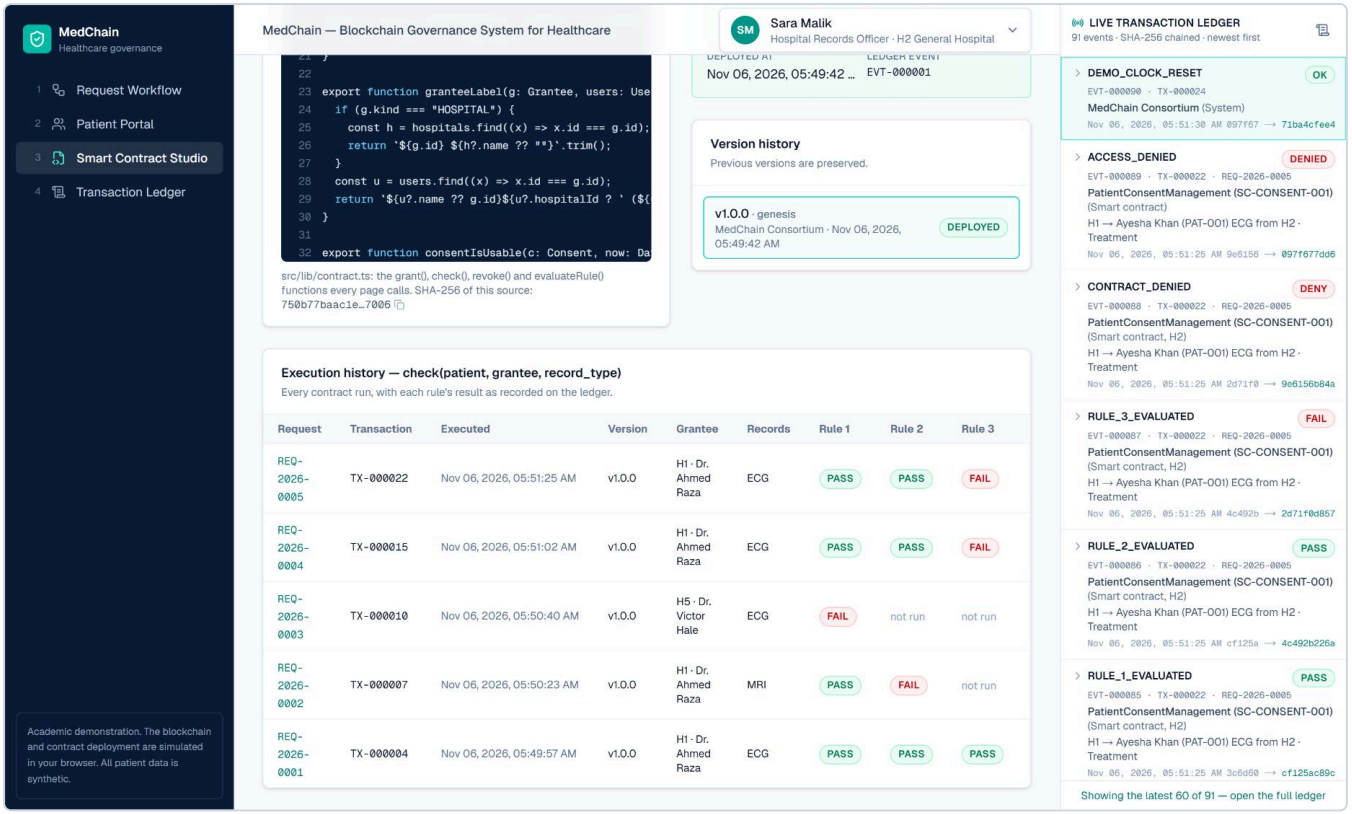


Figure 27. Execution history of check() for every request: result of each rule and the decision.

12. Requirement compliance matrix

Requirement	How it is met	Evidence
Algorithm 08 implemented as a reusable TypeScript module with grant(), check(), revoke()	src/lib/contract.ts; used by every page through the central store	§4, Fig. (Contract Studio)
Three fixed rules R1–R3 evaluated individually with inputs, PASS/FAIL and explanation	evaluateRule(1..3); each result stored and shown with its event ID	§6 Step 4
Any failed rule denies access; no override	check() stops at the first failure; the UI offers no approve/release action after a denial; the store rejects approval unless status is CONTRACT_APPROVED	§7, §8
Step 1 — authentication, credential, hospital, role	Sign-in records AUTHENTICATION_VERIFIED, CREDENTIAL_VERIFIED, HOSPITAL_VERIFIED	§6 Step 1
Step 2 — patient history request with request ID and transaction reference	REQUEST_CREATED; e.g. REQ-2026-0001 / TX-000004	§6 Step 2
Step 3 — source hospital receives and reviews	REQUEST_RECEIVED, REQUEST_REVIEW_STARTED	§6 Step 3
Step 4 — hospital verification and response (approve or reject with reason)	HOSPITAL_VERIFICATION_COMPLETED; HOSPITAL_APPROVED or HOSPITAL_REJECTED	§6 Steps 3 and 5
Step 5 — smart-contract execution	CONTRACT_EXECUTION_STARTED, RULE_1/2/3_EVALUATED, CONTRACT_APPROVED / CONTRACT_DENIED	§6 Step 4, §7
Step 6 — final decision; records released only if rules pass and hospital approves	ACCESS_GRANTED + MEDICAL_RECORD_RELEASED, or ACCESS_DENIED; only requested and consented categories released	§6 Step 6
Step 7 — patient notification and access history	Patient Portal lists every request, hospital, decision and reason	§9
Patient Portal: profile, categories, grant, purpose (Treatment/Research), expiry, active permissions, revoke, history	Patient Portal screen	§8, §9
Revocation immediately affects later checks	Revoked consent → REQ-2026-0004 denied by Rule 3	§8.2
Every meaningful activity recorded; Event ID, Transaction ID, Request ID, timestamp, actor and role, hospital, event type, result, explanation, previous and current hash	recordLedgerEvent() in src/lib/store.tsx	§5, all event tables
currentHash = SHA256(previousHash + payload), chained to the previous event	Web Crypto API; canonical JSON payload; genesis previousHash = 64 zeros	§5, Appendix A
Live ledger updates, transaction history, event details, hash inspection, verification, tampering demo, CSV export	Right-side live ledger; Transaction Ledger screen	§10
Tampering detected	Altered event detected at its exact position	§10
Smart Contract Studio: algorithm, rules, functions, ID, version, hash, execution history, simulated deployment	SC-CONSENT-001 v1.0.0, hash d51a7861d350c1e5…	§11
Roles: Patient, Physician, Records Officer, Hospital Administrator, Auditor	One demo user per role (table in §3)	§3
Sample data: Ayesha Khan (PAT-001); H1–H5; consent H1, LAB+ECG, not MRI, Treatment, 31 Dec 2026, ACTIVE	Seed data, shown in the Patient Portal and in SEED_CONSENT_LOADED	§3
Scenarios 1–6 executed by the real algorithm	Approved ECG; MRI (R2); H5 (R1); revoked (R3); expired (R3); tampering	§6–§10

Requirement	How it is met	Evidence
Next.js, TypeScript, Tailwind, central state, localStorage, Web Crypto SHA-256	Next.js 16 App Router; React Context store; persisted after each committed action	§2

13. Test results summary

Scenario	Request	Rules	Outcome	Records released
1. Successful ECG request	`REQ-2026-0001` `TX-000004`	R1 PASS, R2 PASS, R3 PASS	RELEASED All three contract rules passed and the source hospital approved.	REC-001-ECG
2. MRI outside consent	`REQ-2026-0002` `TX-000007`	R1 PASS, R2 FAIL	DENIED Denied: MRI imaging is not in the patient's consent scope (LAB, ECG).	none
3. Unauthorized hospital H5	`REQ-2026-0003` `TX-000010`	R1 FAIL	DENIED Denied: Dr. Victor Hale's credential is not verified; H5 External Hospital is not registered, not an active registered hospital.	none
4. Revoked consent	`REQ-2026-0004` `TX-000015`	R1 PASS, R2 PASS, R3 FAIL	DENIED Denied: consent CON-0001 is REVOKED (revoked Nov 06, 2026, 05:50:48 AM).	none
5. Expired consent	`REQ-2026-0005` `TX-000022`	R1 PASS, R2 PASS, R3 FAIL	DENIED Denied: consent CON-0002 expired on Nov 13, 2026, 11:59:59 PM.	none
6. Ledger tampering	—	—	Verification failed at the altered event; passed again after restore	—

All outcomes above come from the same algorithm, run on the data at the time of each request. No outcome is hard-coded.

Limitations (simulated parts)

- The ledger runs in one browser and is persisted in localStorage; there is no distributed network.
- Credential "signatures" are SHA-256 proofs over the credential content, standing in for real digital signatures.
- Contract deployment is simulated and produces a receipt but no real on-chain transaction.

Appendix A — Full hash chain of the approved transaction (TX-000004)

Each event's previousHash is the currentHash of the event immediately before it in the whole ledger, so the chain links across transactions as well.

Event / Tx / Request	Event type and explanation (as recorded)	Actor (role)	Result	previousHash → currentHash
EVT-000006 TX-000004 REQ-2026-0001	**REQUEST_CREATED** Dr. Ahmed Raza (H1) requested Ayesha Khan's ECG from H2 for Treatment. Reason: Follow-up of recent chest pain; need prior ECG for comparison.	Dr. Ahmed Raza Physician, H1	SUBMITTED	d7cc55d89e896474bd1620c7d29b4d6810cece22bfdbd4ef16a0cc09790f69e0 ↓ **85b6ccf93e473dfa17c18d1f63e0850e5edc3cc600e2777a976379b4e6e87b25**
EVT-000010 TX-000004 REQ-2026-0001	**REQUEST_RECEIVED** H2 received REQ-2026-0001 in its inbox; Sara Malik acknowledged it.	Sara Malik Hospital Records Officer, H2	RECEIVED	fa7d440dfadd7f840ebb78a12ac35c3059a8c47318bb668e130d684e5a961c0f ↓ **645980ce062349ea44fe423f13e3804f60d5bdfab232cc525e21e7493bfe93f9**
EVT-000011 TX-000004 REQ-2026-0001	**REQUEST_REVIEW_STARTED** Sara Malik opened REQ-2026-0001 and started reviewing it.	Sara Malik Hospital Records Officer, H2	UNDER_REVIEW	645980ce062349ea44fe423f13e3804f60d5bdfab232cc525e21e7493bfe93f9 ↓ **85a2c1769e5acd36ff77353bff03ff1a34f228105e994736ac6fe736a9446ef3**
EVT-000012 TX-000004 REQ-2026-0001	**HOSPITAL_VERIFICATION_COMPLETED** Sara Malik verified REQ-2026-0001: patient reference OK, requesting hospital OK, requesting professional OK, requested data held here OK, patient consent on file OK. This is the hospital's check; Algorithm 08 still decides independently.	Sara Malik Hospital Records Officer, H2	VERIFIED	85a2c1769e5acd36ff77353bff03ff1a34f228105e994736ac6fe736a9446ef3 ↓ **1c873ef1931d23a918e5464806f1b7bf073ed5d36f25e8be685eaab9d30b5cf1**
EVT-000013 TX-000004 REQ-2026-0001	**CONTRACT_EXECUTION_STARTED** Sara Malik started PatientConsentManagement v1.0.0: check(PAT-001, u-phy-h1@H1, [ECG]).	Sara Malik Hospital Records Officer, H2	RUNNING	1c873ef1931d23a918e5464806f1b7bf073ed5d36f25e8be685eaab9d30b5cf1 ↓ **2bea1b2e57cbfb3ee90a20fe39a3c2c97067bb24fe9002c15e86e339fea72298**
EVT-000014 TX-000004 REQ-2026-0001	**RULE_1_EVALUATED** Rule 1 (Identity and hospital authorization): Both professionals are signed in with verified credentials, work at active registered hospitals, and hold the required role permissions.	PatientConsent Management (SC-CONSENT-001) Smart contract, H2	PASS	2bea1b2e57cbfb3ee90a20fe39a3c2c97067bb24fe9002c15e86e339fea72298 ↓ **91ddcdc737ac2ceff5c4379f26fe6faec393a74dbb970c42f3d1e969a24841e2**
EVT-000015 TX-000004 REQ-2026-0001	**RULE_2_EVALUATED** Rule 2 (Patient consent and record scope): Consent CON-0001 authorizes H1 City Hospital and covers all requested types (ECG).	PatientConsent Management (SC-CONSENT-001) Smart contract, H2	PASS	91ddcdc737ac2ceff5c4379f26fe6faec393a74dbb970c42f3d1e969a24841e2 ↓ **4612fdba41c06b5d06242d06b669613a4c162aec64dd4232b3335216cad3448f**
EVT-000016 TX-000004 REQ-2026-0001	**RULE_3_EVALUATED** Rule 3 (Purpose, expiry and revocation): Purpose matches and the consent is active, not revoked and not expired.	PatientConsent Management (SC-CONSENT-001) Smart contract, H2	PASS	4612fdba41c06b5d06242d06b669613a4c162aec64dd4232b3335216cad3448f ↓ **5798055008a63aced7a9e5cc72a832c408b57c5d9313ad7a9116d3711d4da158**
EVT-000017 TX-000004 REQ-2026-0001	**CONTRACT_APPROVED** check() returned APPROVE: all three fixed rules passed. The source hospital must now approve or reject.	PatientConsent Management (SC-CONSENT-001) Smart contract, H2	APPROVE	5798055008a63aced7a9e5cc72a832c408b57c5d9313ad7a9116d3711d4da158 ↓ **c24f144df2c550cbad5c71d972573f36fedfb1405dd4190282d0d04de751320d**

Event / Tx / Request	Event type and explanation (as recorded)	Actor (role)	Result	previousHash → currentHash
EVT-000018 TX-000004 REQ-2026-0001	**HOSPITAL_APPROVED** Sara Malik (H2) approved REQ-2026-0001. Contract passed and hospital approved: the permitted records may now be released.	Sara Malik Hospital Records Officer, H2	APPROVED	c24f144df2c550cbad5c71d972573f36fedfb1405dd4190282d0d04de751320d ↓ **007a3f8cc444fefa3db7551e8a46b96c9a60c88e91933712da760103e1b9672d**
EVT-000019 TX-000004 REQ-2026-0001	**ACCESS_GRANTED** Access granted: all three contract rules passed and the source hospital approved.	Sara Malik Hospital Records Officer, H2	GRANTED	007a3f8cc444fefa3db7551e8a46b96c9a60c88e91933712da760103e1b9672d ↓ **3ac5313253d16ada02a48c09f1cc3e357a181ab4bda8916c2bcaa534a10d357c**
EVT-000020 TX-000004 REQ-2026-0001	**MEDICAL_RECORD_RELEASED** Released REC-001-ECG (ECG) from H2 to H1. Nothing outside the approved categories was released.	Sara Malik Hospital Records Officer, H2	RELEASED	3ac5313253d16ada02a48c09f1cc3e357a181ab4bda8916c2bcaa534a10d357c ↓ **48b8140f9abb073b0036b3301c02cf875f8c7ec1c7b1dc501ea755c75adc0457**

Appendix B — Contract code as deployed (SC-CONSENT-001 v1.0.0)

```
contract PatientConsentManagement {
  id      = "SC-CONSENT-001";
  version = "1.0.0";

  // ----- governance parameters -----
  param maxConsentDays    = 365;
  param emergencyPathway  = false;

  struct Consent { patient; grantee; allowedScope[]; purpose; expiry; status; }

  // Algorithm 08 — grant(patient, grantee, scope, purpose, expiry)
  function grant(patient, grantee, scope, purpose, expiry) {
    require(caller == patient && did.isRegistered(patient));
    require(scope.length > 0);
    require(now < expiry && expiry <= now + days(maxConsentDays));
    for (c in consents.active(patient, grantee)) c.status = SUPERSEDED;
    consents.add(Consent(patient, grantee, scope, purpose, expiry, ACTIVE));
    emit ConsentGranted(patient, grantee, scope, purpose, expiry);
  }

  // Algorithm 08 — check(patient, grantee, record_type)
  // grantee = requesting professional and their hospital; the request supplies purpose and processor.
  function check(patient, grantee, record_type) returns (Decision) {
    consent = consents.latestFor(patient, grantee);   // newest ACTIVE, else newest of any status
    if (!R1_IdentityAndHospital(grantee, processor))                return deny(R1);
    if (!R2_ConsentAndScope(consent, grantee, record_type))         return deny(R2);
    if (!R3_PurposeExpiryRevocation(consent, request.purpose, now)) return deny(R3);
    emit ContractDecision(request.id, APPROVE);
    return APPROVE;   // eligible: consensus, then the source hospital's final approval
  }

  // Algorithm 08 — revoke(patient, grantee)
  function revoke(patient, grantee) {
    require(caller == patient);
    require(consents.active(patient, grantee).length > 0);
    for (c in consents.active(patient, grantee)) { c.status = REVOKED; c.revokedAt = now; }
    emit ConsentRevoked(patient, grantee, now);
  }

  // A failed rule always denies. There is no override function.

  // ===== FIXED BASELINE RULES — immutable, no user or administrator can edit or override them =====
  rule R1_IdentityAndHospital(grantee, processor) returns (bool) {
    return grantee.authenticated && credentials[grantee].verified
        && hospitals[grantee.hospital].status == ACTIVE
        && RBAC.has(grantee.role, "request:create")
        && processor.authenticated && credentials[processor].verified
        && RBAC.has(processor.role, "request:process")
        && hospitals[request.sourceHospital].status == ACTIVE
        && processor.hospital == request.sourceHospital;
  }

  rule R2_ConsentAndScope(consent, grantee, record_type) returns (bool) {
    return consent != null
        && (consent.grantee == grantee.hospital || consent.grantee == grantee)
        && all(t IN consent.allowedScope for t in record_type);
  }

  rule R3_PurposeExpiryRevocation(consent, purpose, now) returns (bool) {
    return purpose == consent.purpose
        && consent.status == ACTIVE        // not revoked or superseded
        && now < consent.expiry;           // not expired
  }
}
```